

ANALOG & DIGITAL CIRCUITS

Lectures 11,12

Computer Science Department, TUCN

80C51 MICROCONTROLLER FAMILY

Introduction

Memory addressing

Interrupts system

Timer system registers

Serial communication

Introduction

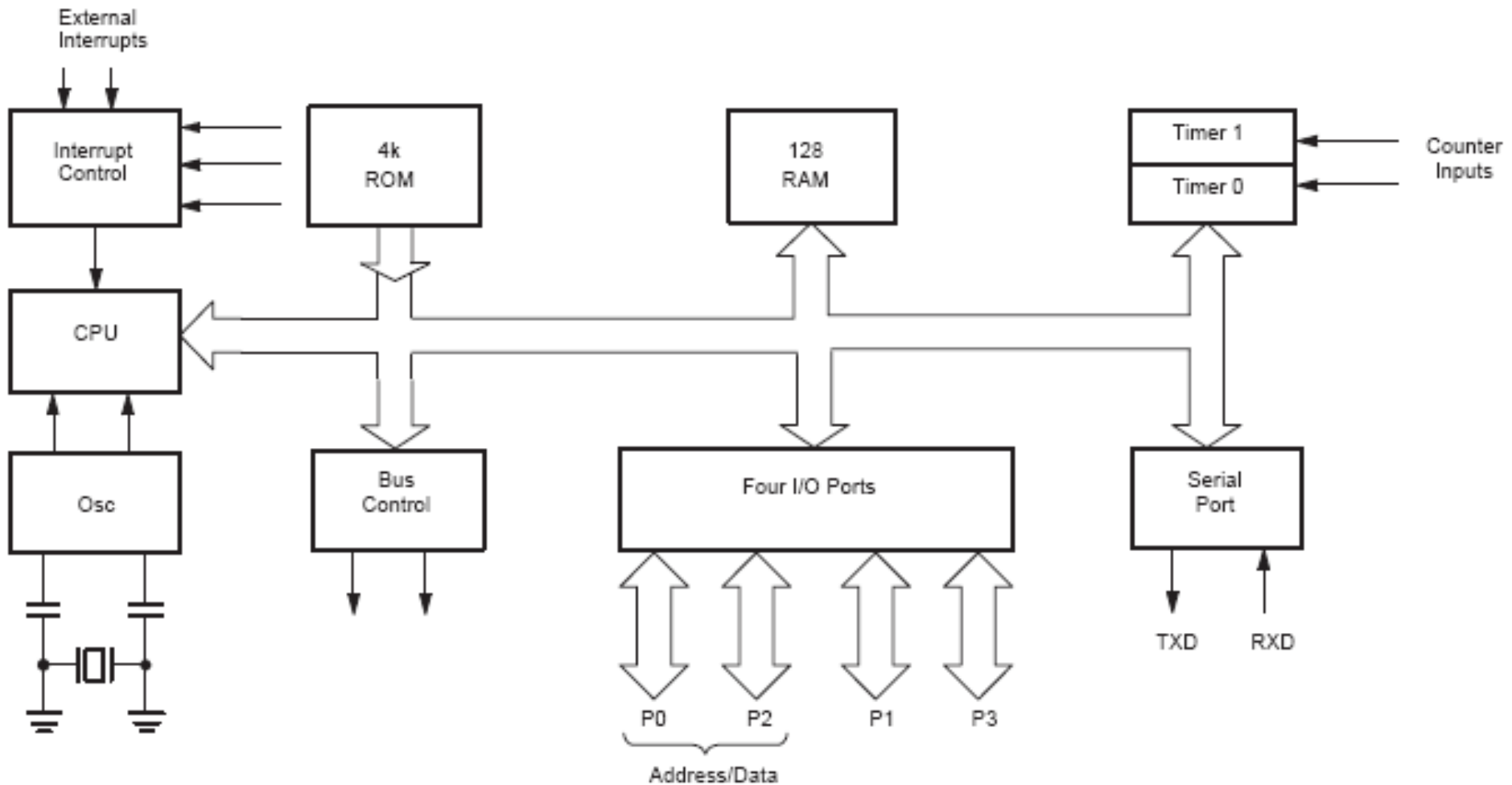
Microcontrollers are embedding all circuits needed by any general-use computer:

- Boolean processor
- Memory
- Counters/timers
- Interrupt system
- Input/output system

80C51 Features

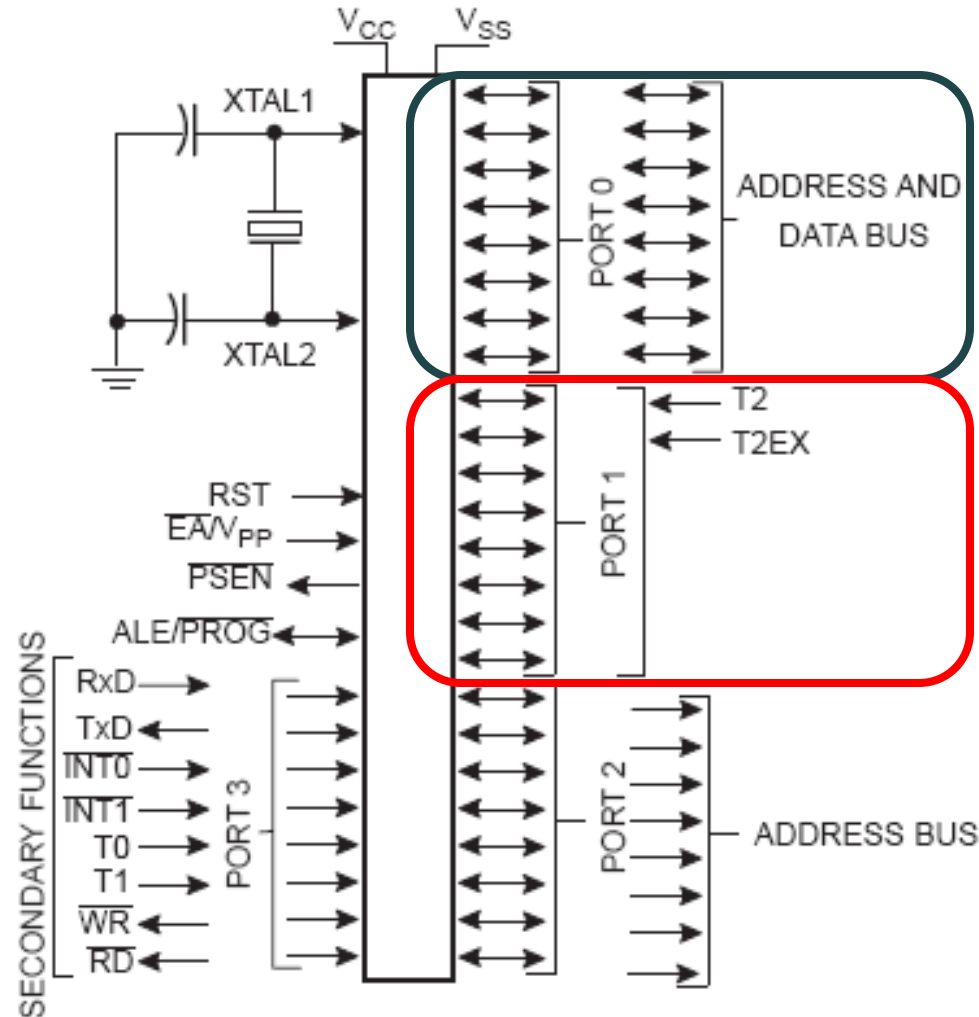
- 8051 Central Processing Unit
 - 4k*8 ROM (codes)
 - 128*8 RAM (variables)
 - 3*16-bit counter/timers
 - Boolean processor
- External memory addressing capability
 - 64k*8 ROM (program)
 - 64k*8 RAM (data)
- 6 interrupts with 2 level priority
- 4*8-bit I/O ports
- Full-duplex UART
- Asynchronous port reset
- Data, address and control buses

Block diagram



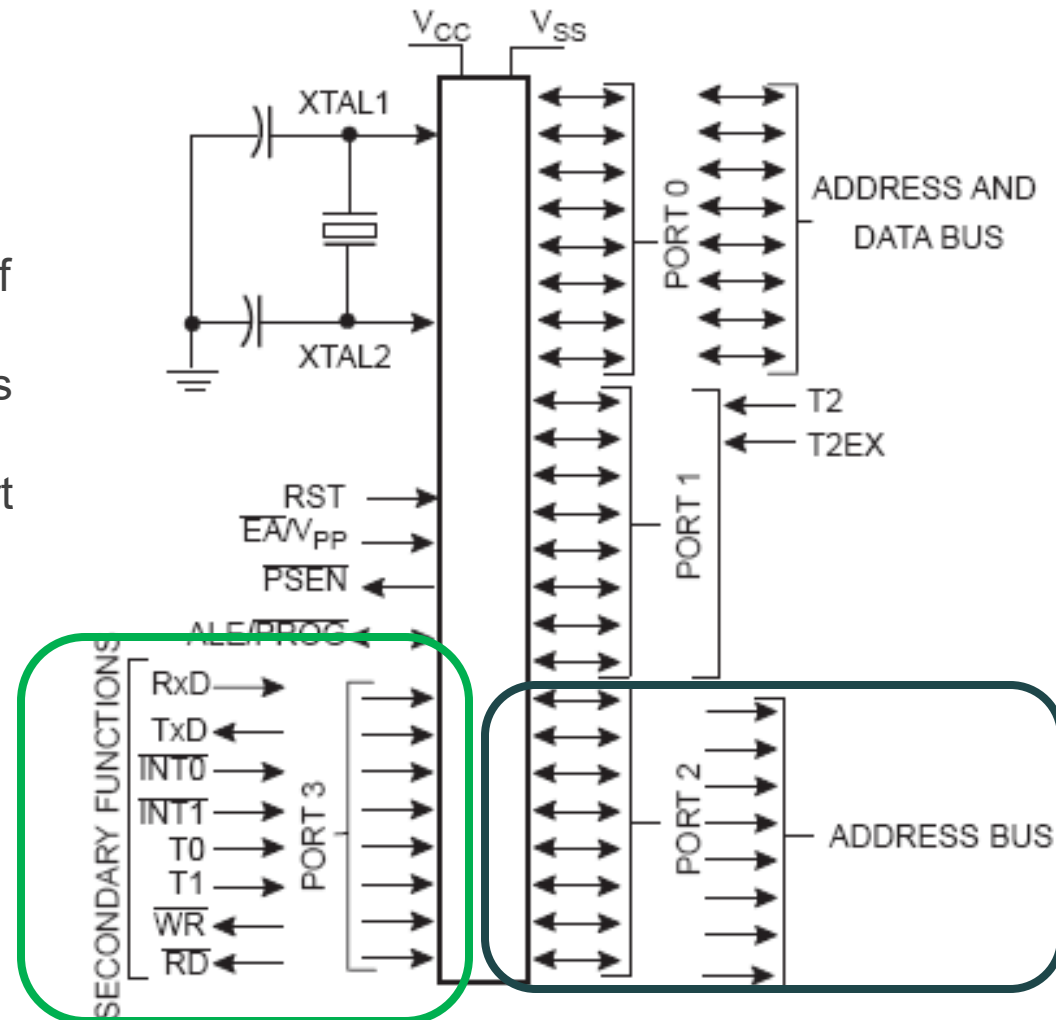
Logic symbol and pin description

- V_{CC} - Power Supply + V_{SS} - 0 V reference
- Each port has 8 pins; All ports are bidirectional; All the ports are configured as input ports on Reset
- They have a main function and alternative function:
- **P0**: bidirectional port; can be used for address (low-order address) or data (accesses to external program and data memory)
- It does not have an internal pull-up resistor. An external pull-up resistor is needed when Port 0 is defined as an output port.
- **P1**: bidirectional port + 2 extra functions (Timer 2 is a 16-bit Timer/Counter which can operate as either an event timer or an event counter)



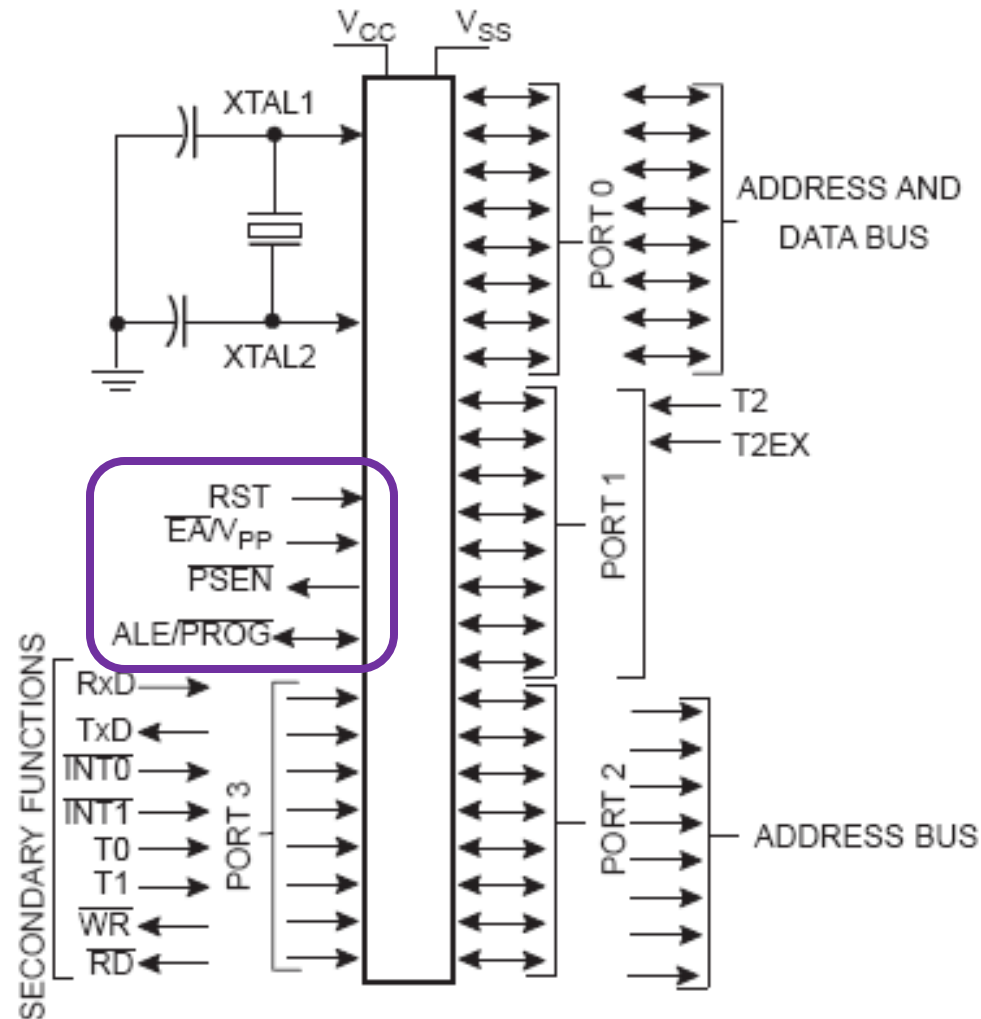
Logic symbol and pin description

- **P2**: bidirectional port; can be used for address (high-order address)
- When the 8051 wants to access external memory, the address for the memory generates due to Port 0 and Port 2. We get the lower half of the address from Port 0 and the upper half from Port 2. This is done using **ALE** pulses, which help to latch the address to the external bus. Once done, the Port 0 goes back to being an input port to read data from that memory.
- **P3**: bidirectional port; plus several functions:
 - **RxD, TxD** – for serial communication
 - **INT0, INT1** – external interrupts
 - **T0, T1** – external timers
 - **RD, WR** – to work with External data memory



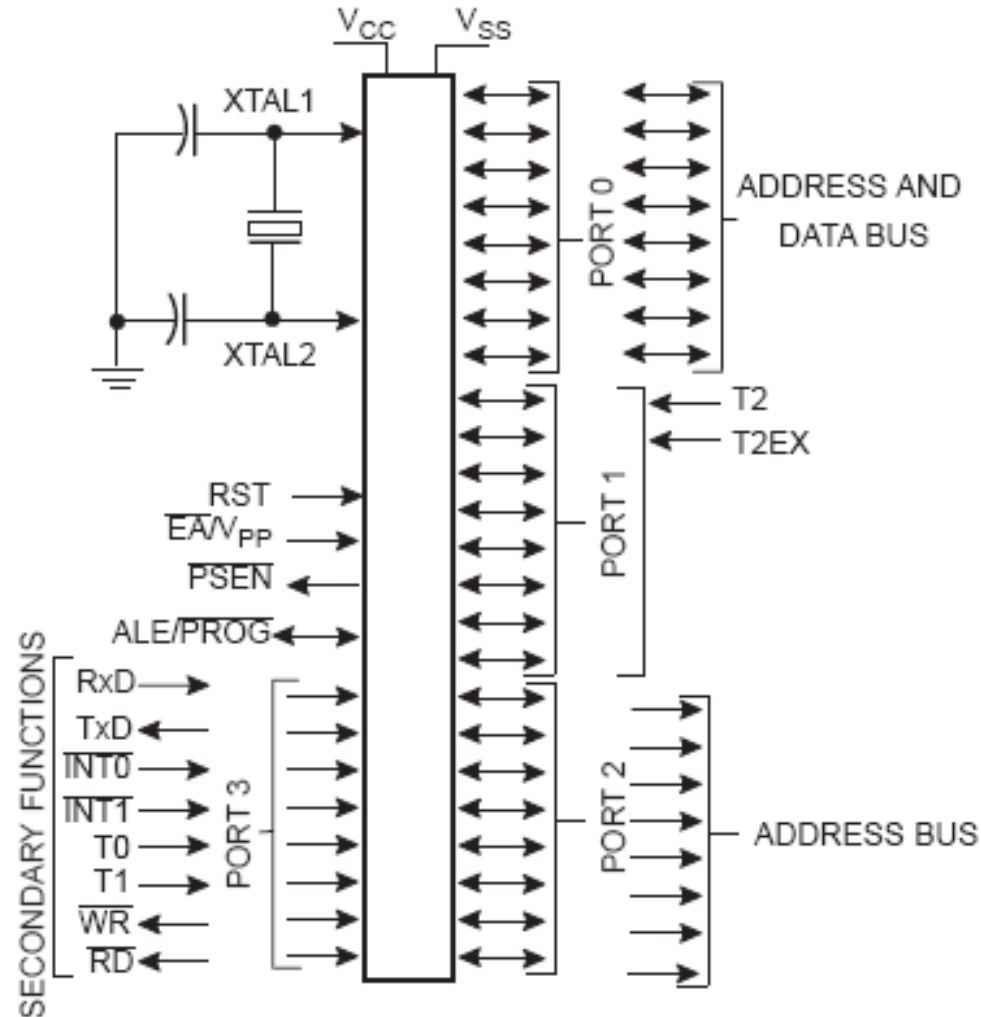
Logic symbol and pin description

- **RST**
- **ALE/PROG:** ALE output pulse is used for latching the low byte of the address during an access to external memory. PROG pin is the program pulse input during EPROM programming.
- **PSEN** – for External program memory read signal
- **EA[\] / V_{PP}**
 - If EA[\] is externally held low, the microcontroller executes the entire code from external program memory. PSEN is used for every instruction fetch
 - If EA[\] is externally held high, the microcontroller executes the code located at memory locations 0000H to 0FFFH from the internal ROM.

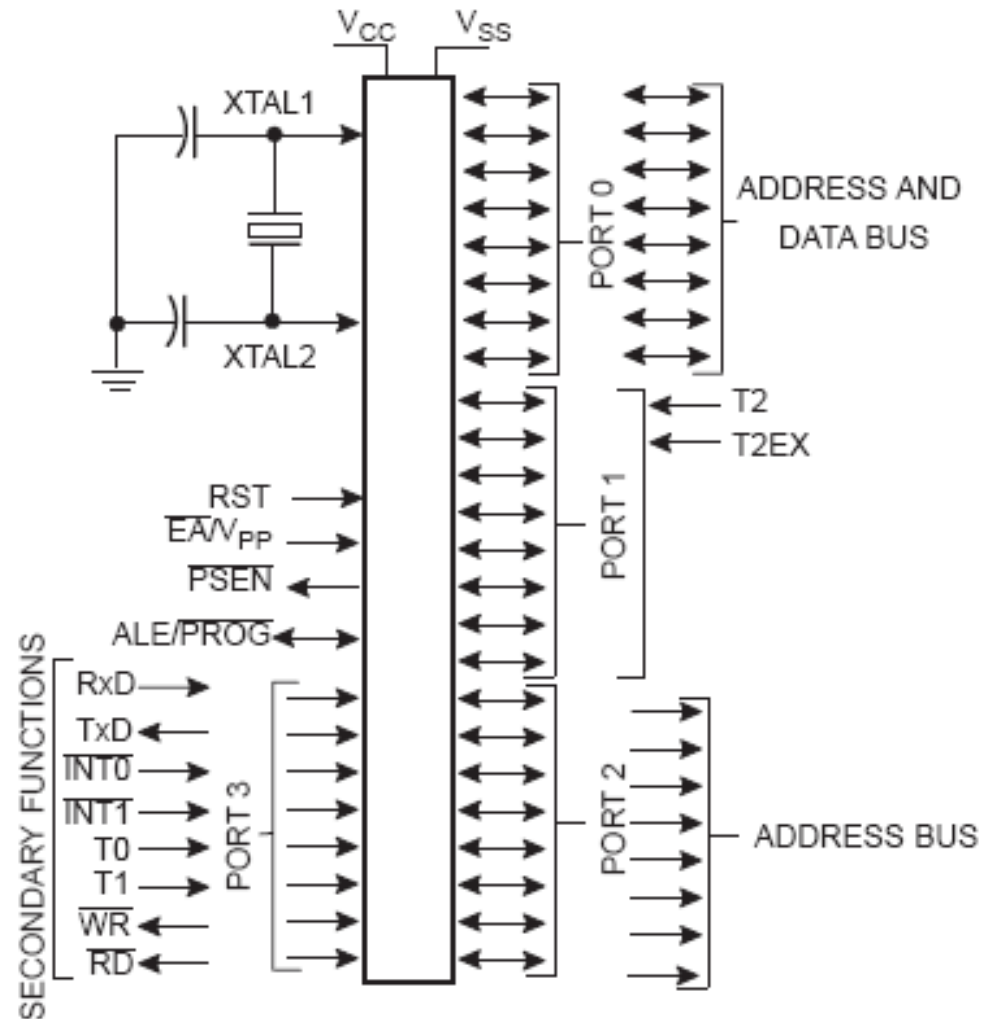


Logic symbol and pin description

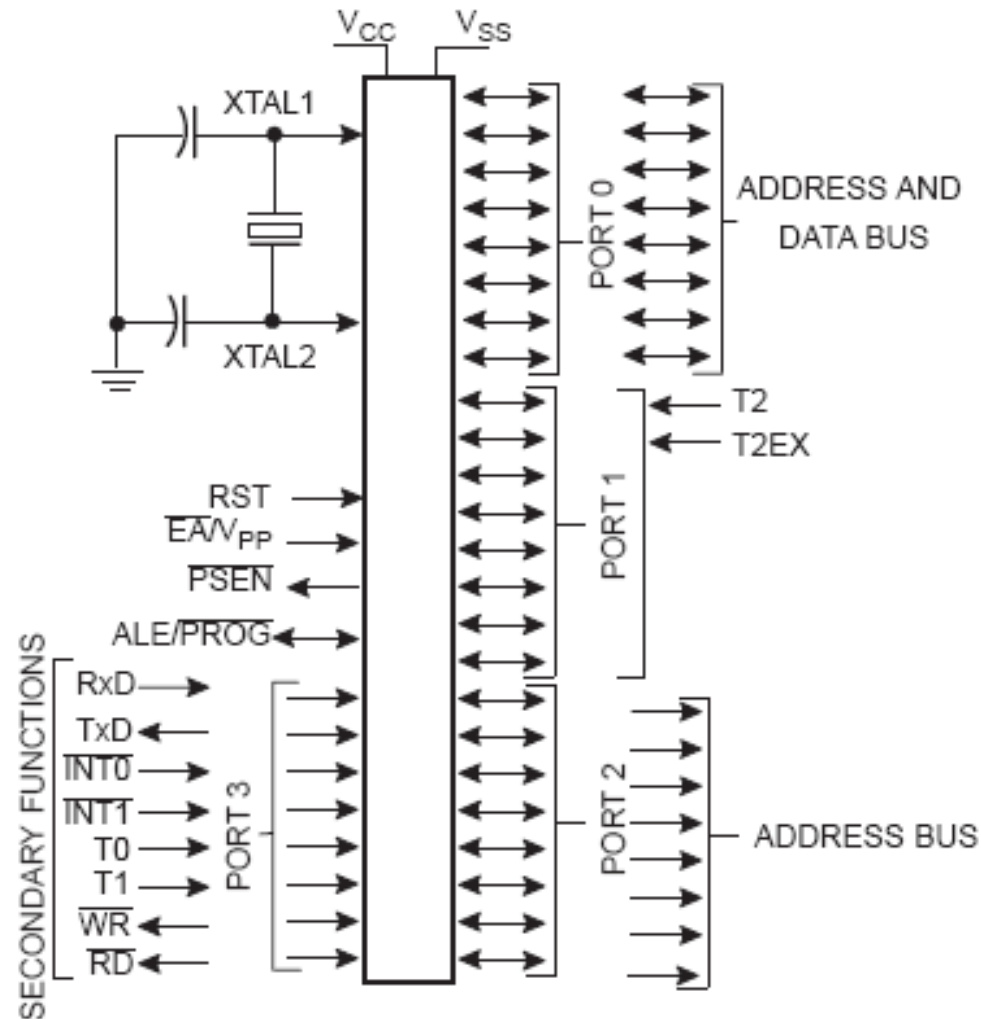
- **V_{SS}, Input, Ground:** 0 V reference
- **V_{CC}, I, Power Supply:** Power supply voltage
- **P0.0–0.7, I/O, Port 0:** Can have the function of 8-bit open-drain bidirectional port with Schmitt trigger inputs. Pins that have 1s written to them are floating and can be used as high-impedance inputs. Can have the function of multiplexed low-order address and data bus during accesses to external program and data memory using internal pull-ups.
- **P1.0–P1.7, I/O, Port 1:** Can have the function of 8-bit bidirectional port with internal pull-ups and Schmitt trigger inputs. Pins that have 1s written to them are pulled high by the internal pull-ups and can be used as inputs. Also, two pins can have alternate functions.
- **T2, I/O, (P1.0):** Timer/Counter 2.
T2EX, I, (P1.1): Timer/Counter 2 with capture function.



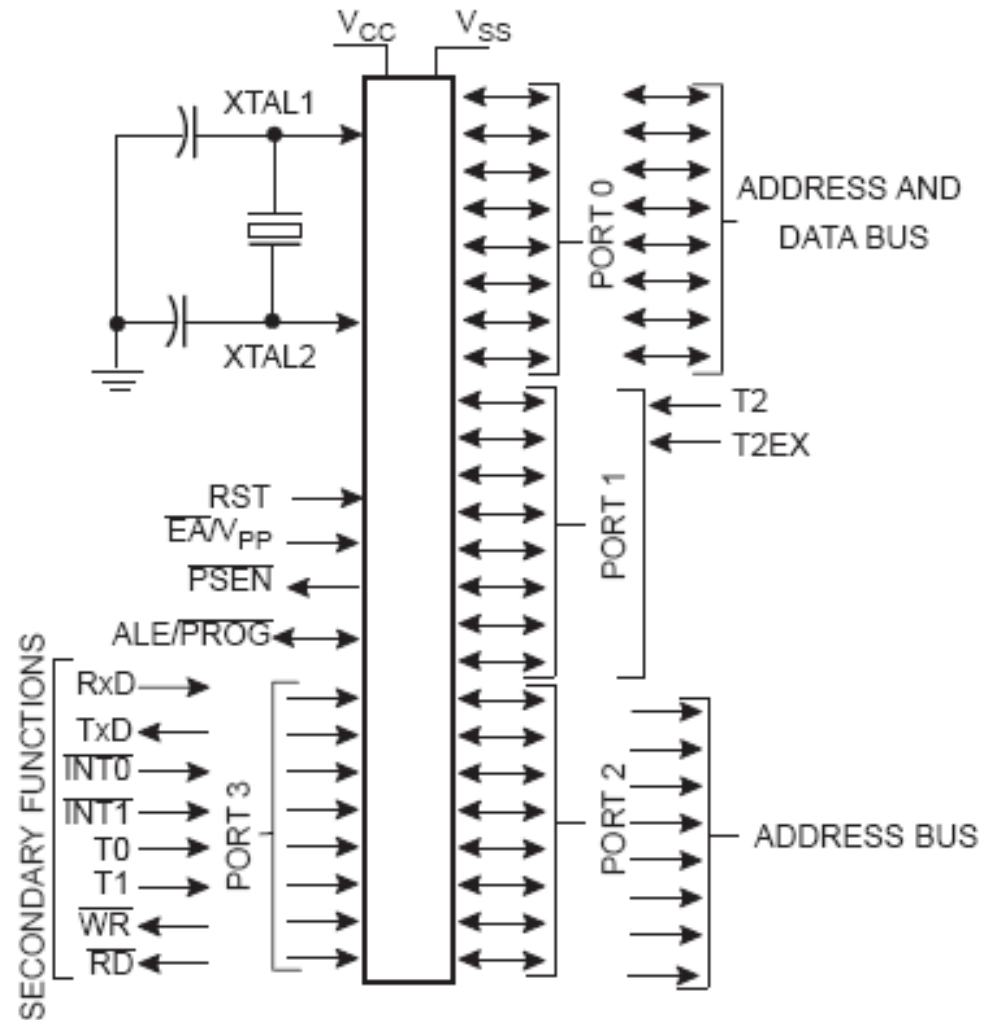
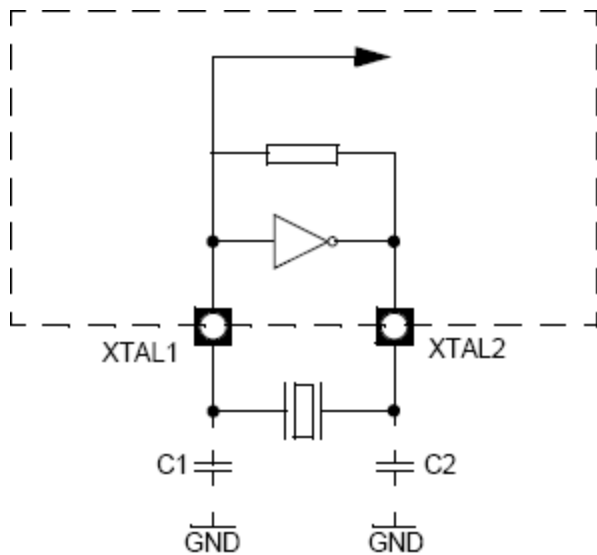
- P2.0–P2.7, I/O, Port 2:** Can have the function of 8-bit bidirectional port with internal pull-ups and Schmitt trigger inputs. Pins that have 1s written to them are pulled high by the internal pull-up resistors and can be used as inputs. As alternate function, generates the high-order address byte during accesses to external program and data memory.
- P3.0–P3.7, I/O, Port 3:** Can have the function of 8-bit bidirectional port with internal pull-ups and Schmitt trigger inputs. Port 3 pins that have 1s written to them are pulled high by the internal pull-ups and can be used as inputs. Also, each pin can have alternate function.
 - RxD, I, (P3.0):** Serial input port.
 - TxD, O, (P3.1):** Serial output port.
 - INT0, I, (P3.2):** External interrupt.
 - INT1, I, (P3.3):** External interrupt.
 - T0, I, (P3.4):** Timer 0 external input.
 - T1, I, (P3.5):** Timer 1 external input.
 - WR, O, (P3.6):** External data memory write strobe.
 - RD, O, (P3.7):** External data memory read strobe.



- **RST, I, Reset:** A high on this pin resets the microcontroller.
- **ALE/PROG $\overline{\text{A}}$, I/O, Address Latch Enable/Program Pulse:** ALE output pulse is used for latching the low byte of the address during an access to external memory. PROG pin is the program pulse input during EPROM programming.
- **PSEN, O, Program Store Enable:** External program memory read strobe.
- **EA $\overline{\text{A}}$ / V_{PP}, I, External Access Enable/Programming Supply Voltage:** If EA $\overline{\text{A}}$ is externally held low, the microcontroller executes the entire code from external program memory. If EA $\overline{\text{A}}$ is externally held high, the microcontroller executes the code located at memory locations 0000H to 0FFFH from the internal ROM. V_{PP} pin receives the programming supply voltage during EPROM programming.

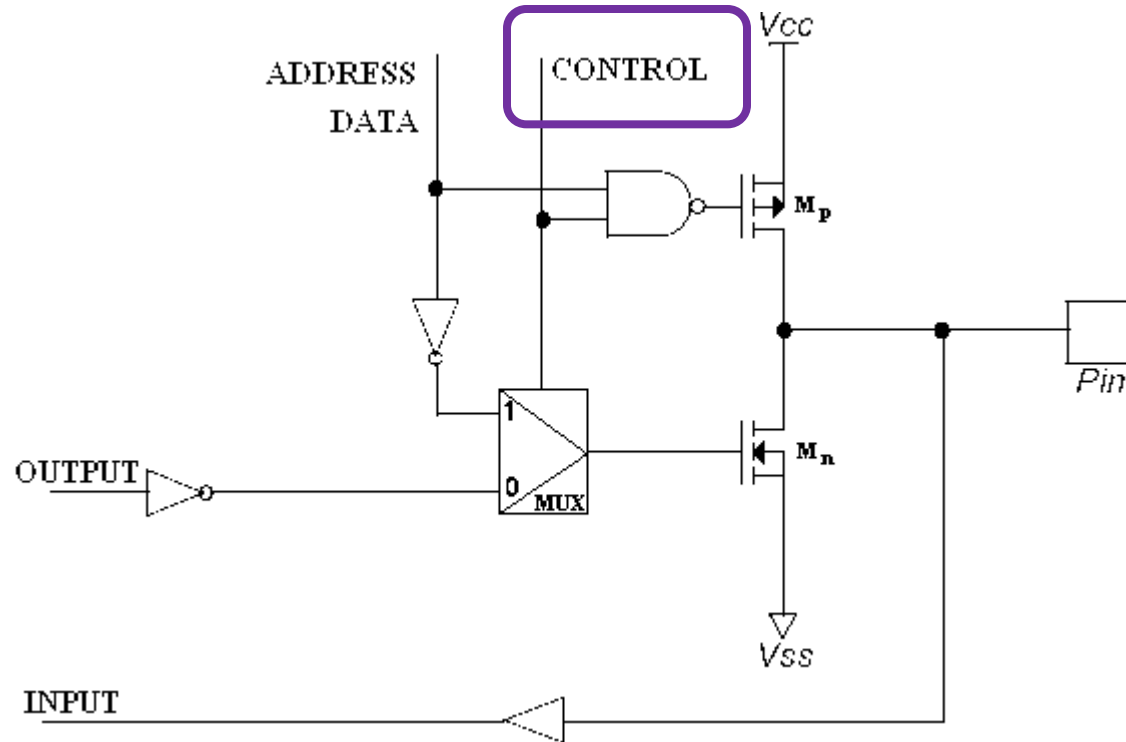


- **XTAL1, I, Crystal 1:** Input to the inverting oscillator amplifier and clock generator circuit.
- **XTAL2, O, Crystal 2:** Output from the inverting oscillator amplifier.



Port 0 layout

- If $\text{CONTROL}=1$, P0 is used for address and data bus using internal pull-ups. The output of the NAND gate isn't influenced by the CONTROL signal, so, the state of the M_p transistor is determined only by the logical state of the address or data transmitted.



- If $\text{CONTROL}=0$, P0 is an open-drain bidirectional port. The output of the NAND gate is 1, so, the M_p transistor is in cut-off stage, determining the open-drain configuration of the port. So, if the port is used for output, external pull-ups are used. If the port is used as an input port, logical value 1 is software set to the pin, so, the M_n transistor is in cut-off stage and the input floats.

80C51 MICROCONTROLLER FAMILY

Introduction

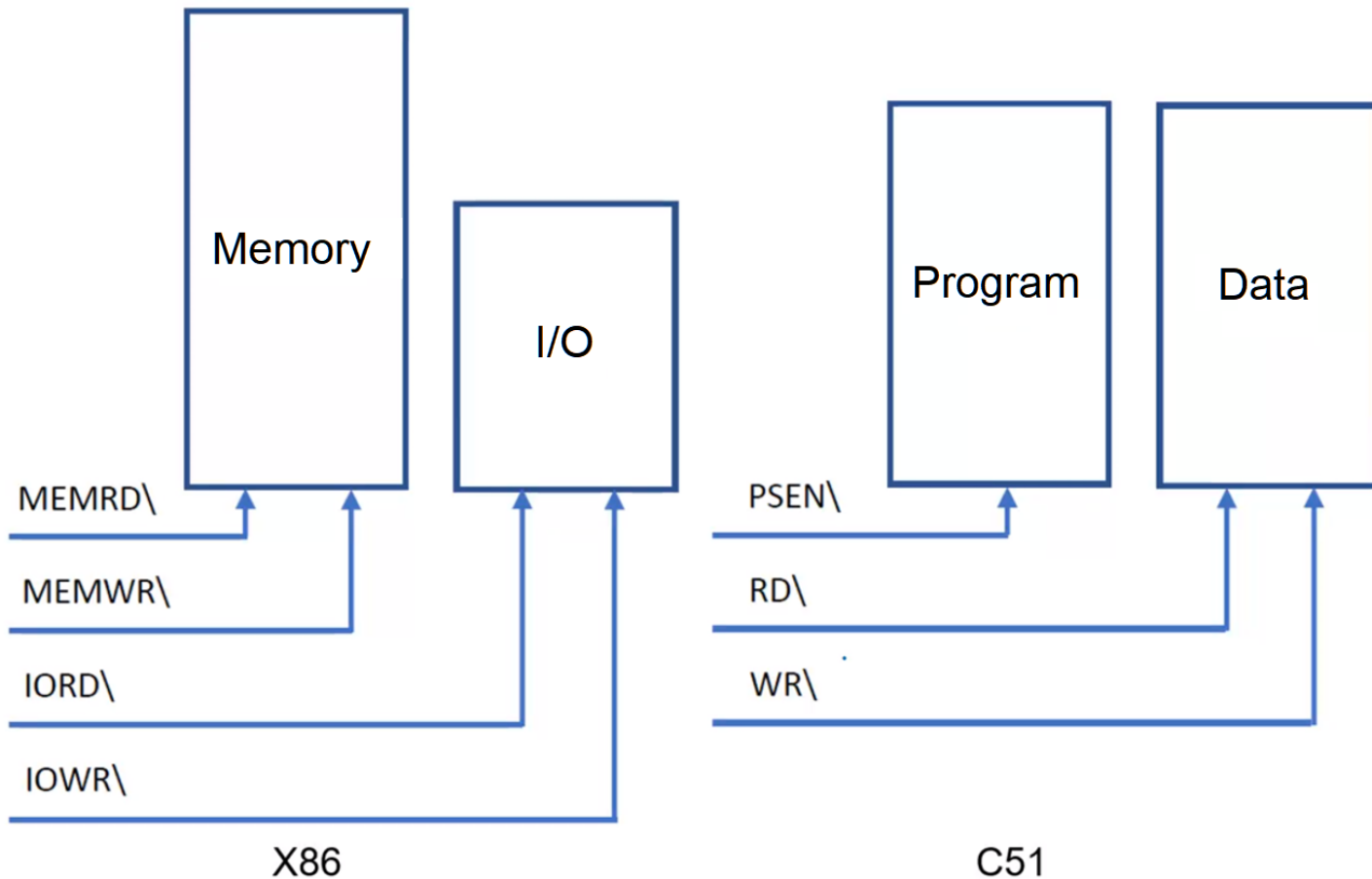
Memory addressing

Interrupts system

Timer system registers

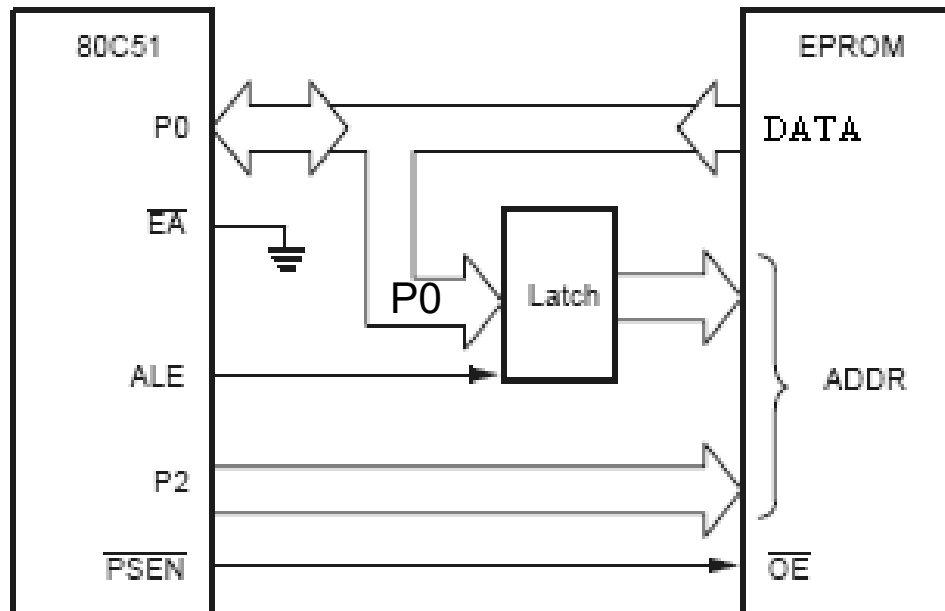
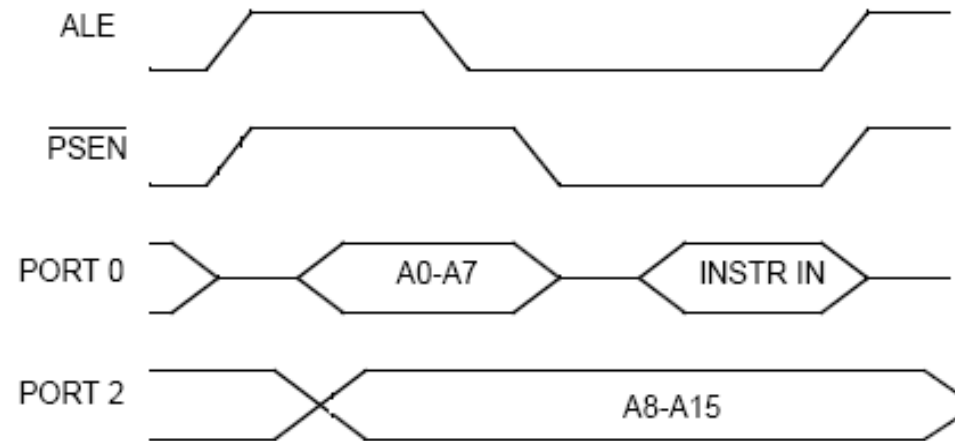
Serial communication

Memory addressing: x86 vs C51



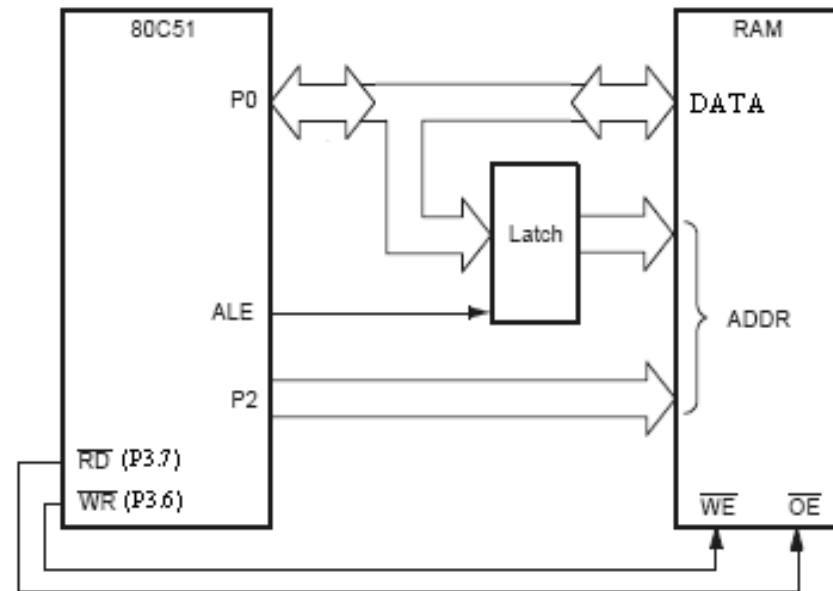
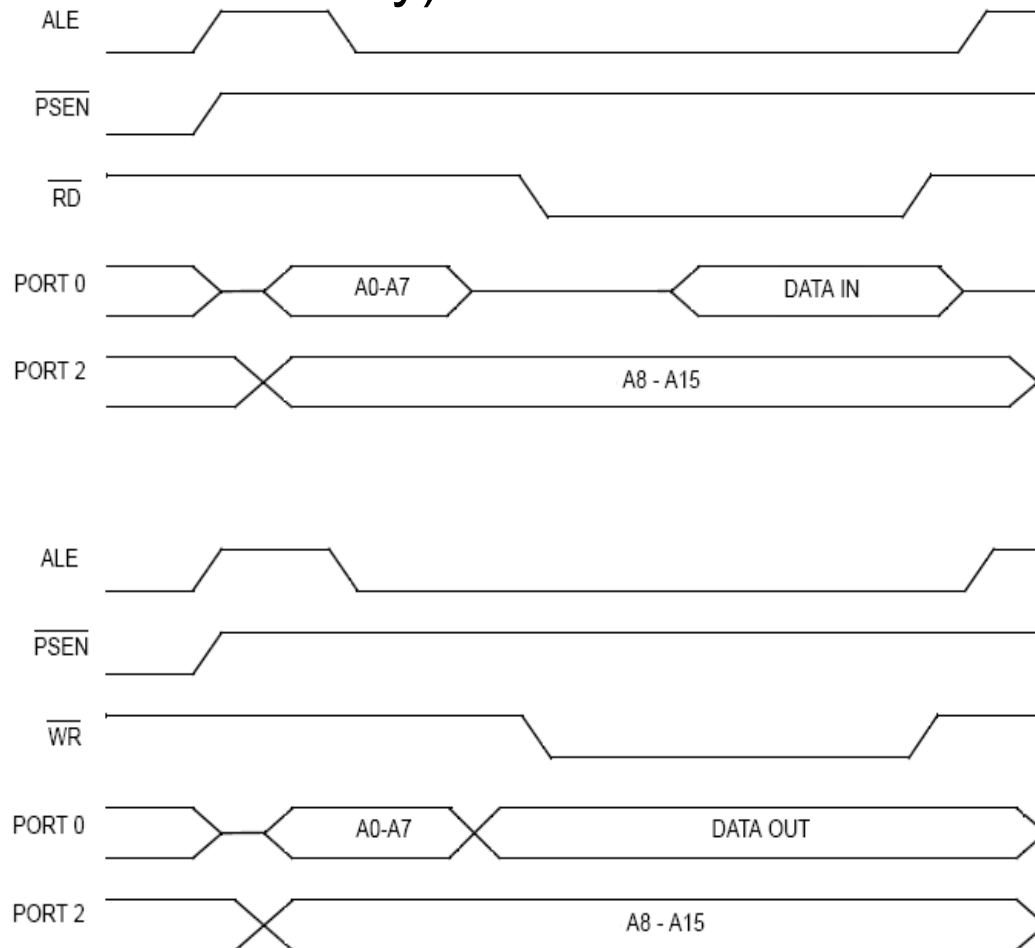
External program memory read cycle and configuration

- Address is transmitted first through **P0** and **P2** ports on the address bus. Because P0 port changes its function becoming data port, A0-A7 is latched when the **ALE** signal becomes active. **PSEN** signal becomes active and the information in the program memory is transmitted on the data bus being received through **P0** port.
- **EA** is externally held low, the microcontroller executes the entire code from external program memory.



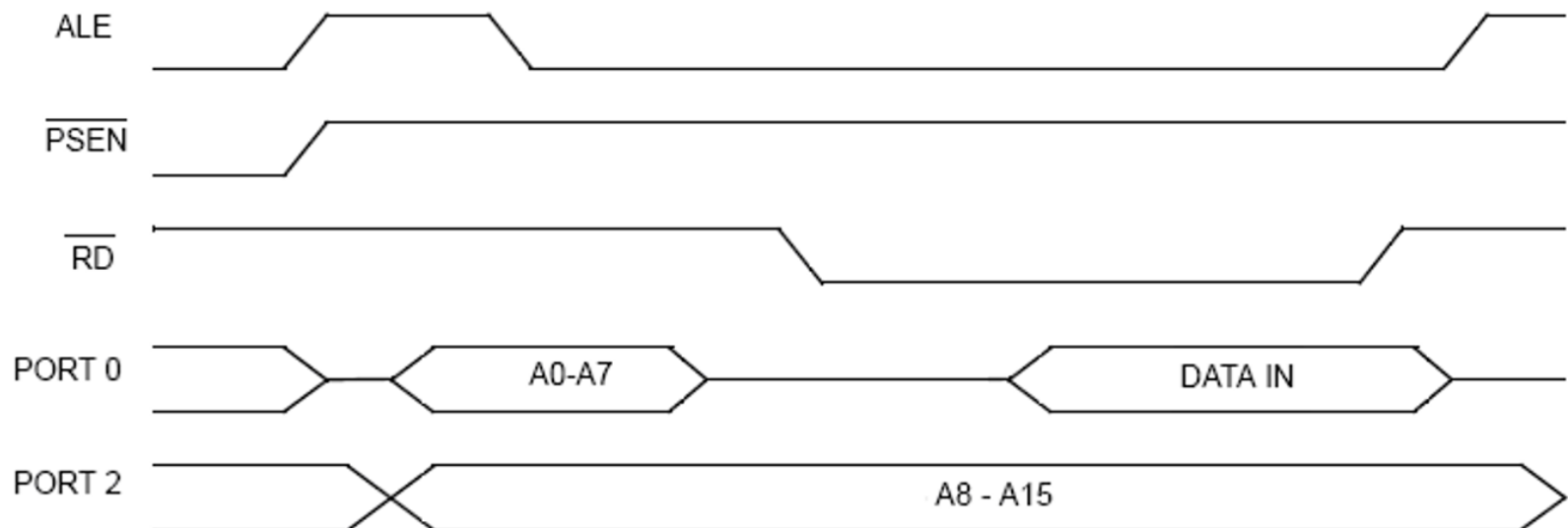
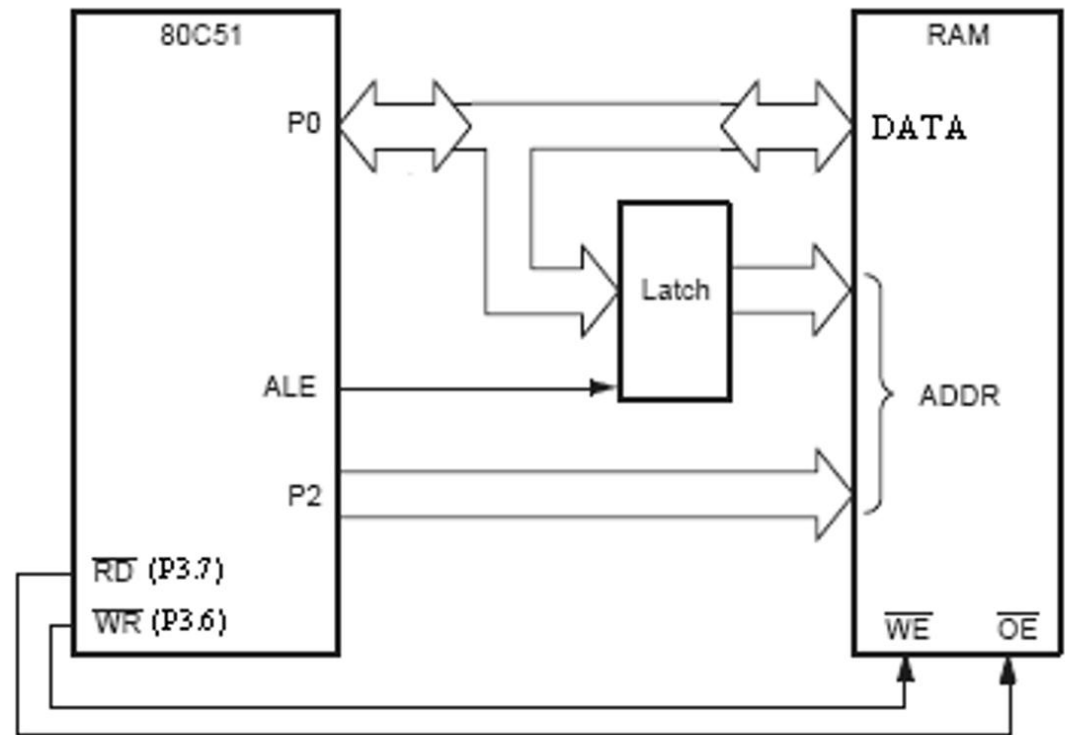
External data memory read and write cycles

- Similar with previous
- PSEN is replaced by **RD** (to read from memory) and **WR** (to write to memory)



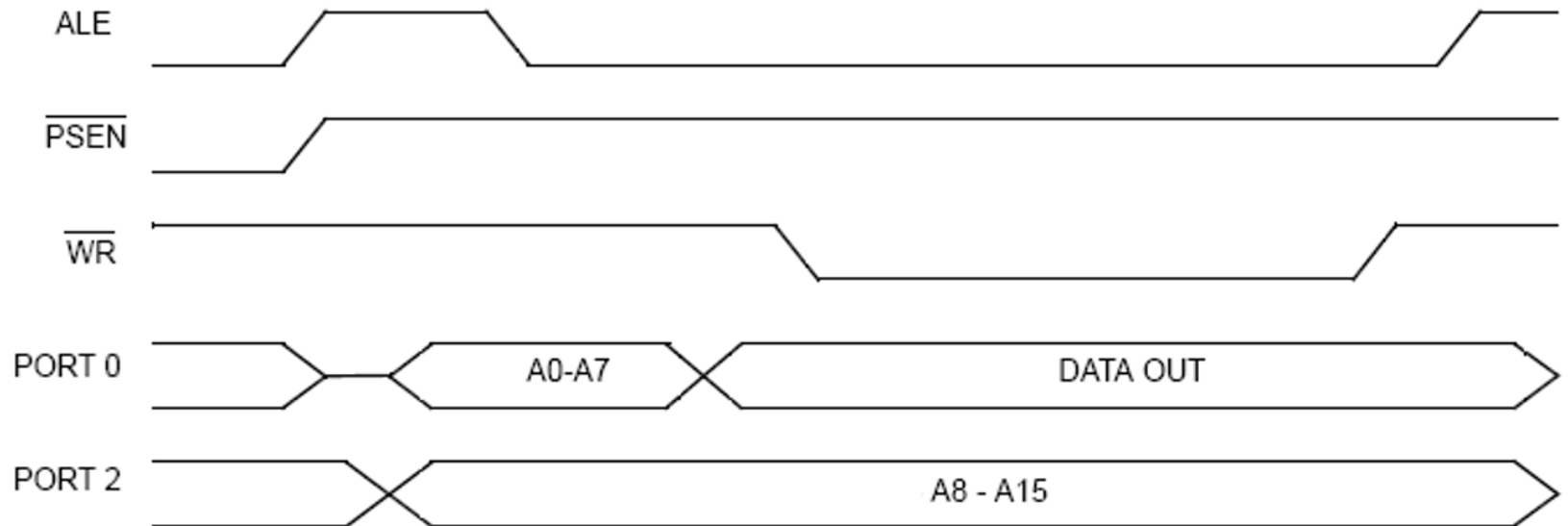
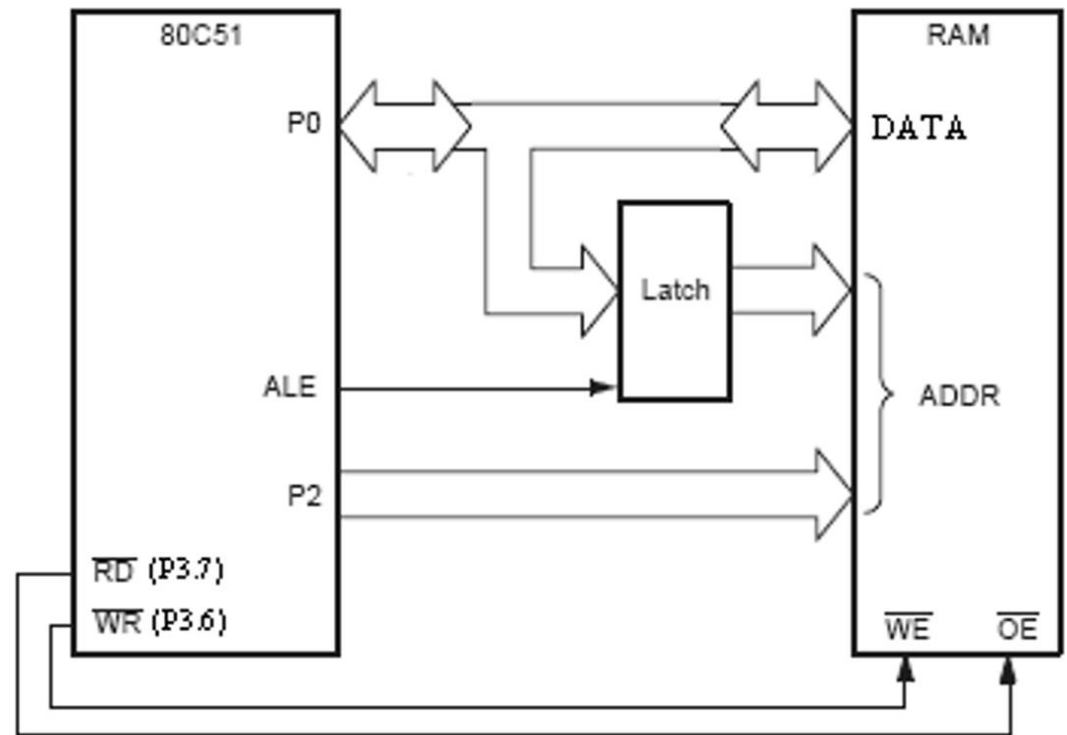
Memory read cycle:

1. Address sent through **P0** and **P2**; latch P0 with **ALE**
2. **RD** active: info from data memory sent to microcontroller on port P0



Memory write cycle:

1. Address sent through **P0** and **P2**; latch P0 with **ALE**
2. Data sent on the data bus using P0 and when **WR** is active, data is stored in the external memory

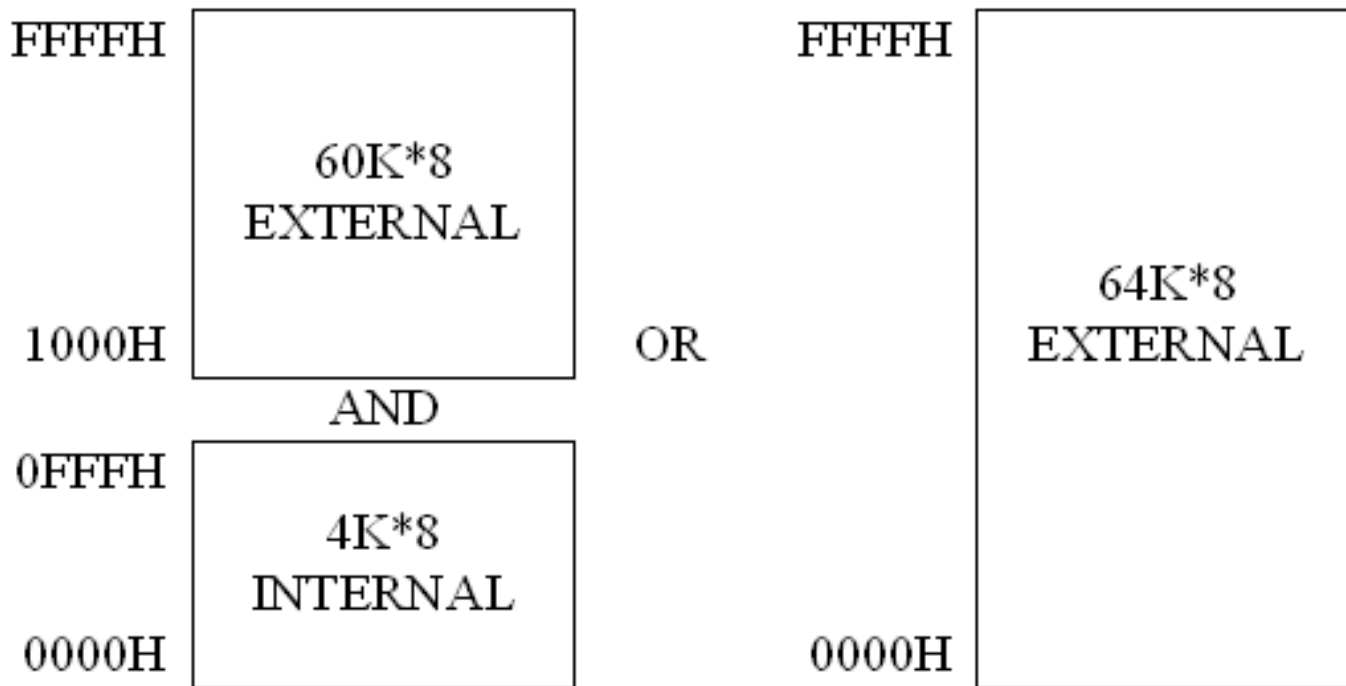


Memory organization

- 80C51 has separate address spaces for program and data memory.

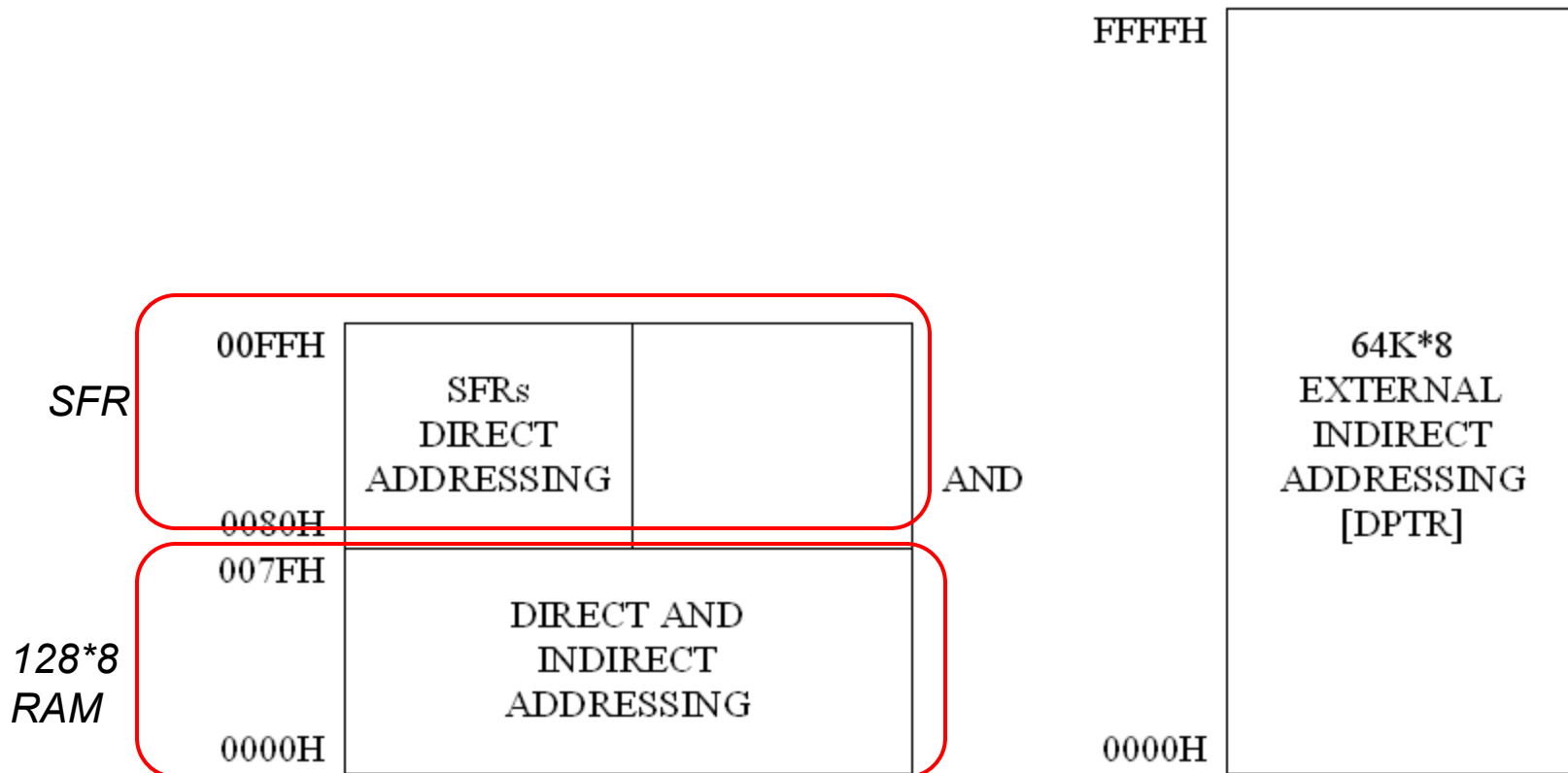
PROGRAM MEMORY

- The program memory can be up to 64K*8 long.
 - If EA\ is externally held high, the lower 4K*8 addresses reside on chip.
 - If EA\ is externally held low, the entire memory is external.



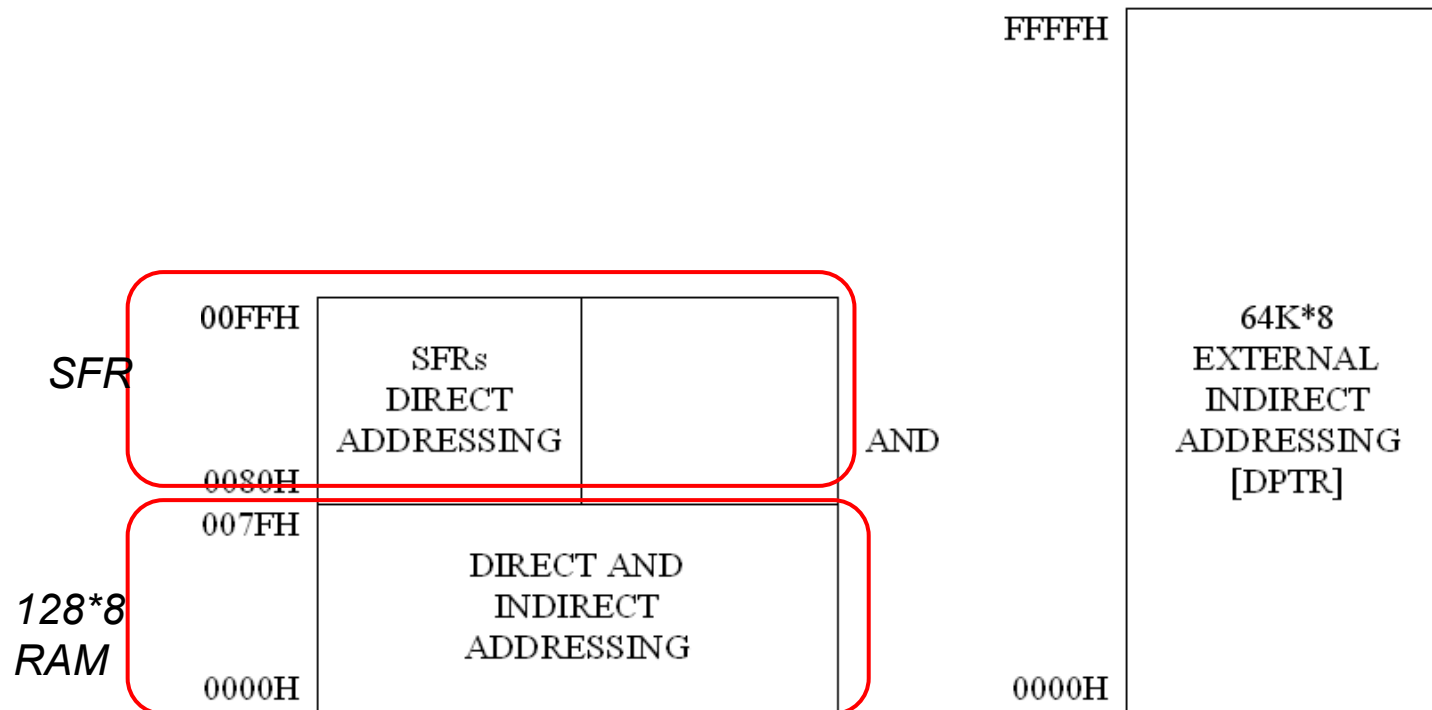
DATA MEMORY

- 80C51 can access both internal and external data memory.
- 80C51 has **128*8 internal RAM** plus a number of **Special Function Registers (SFRs)**. The lower 128*8 RAM addresses can be accessed either by direct or indirect addressing. SFRs can be accessed only by direct addressing. Direct addressing specifies the address through a number, indirect addressing specifies the address through a register.
- External data memory (RAM) can be up to 64K*8 long.



DATA MEMORY

- Direct addressing specifies the address through a number (address of the data is written directly in the instruction)
 - Ex: MOV A, 27H
(Move the value stored at address 27H into accumulator A)
- Indirect addressing specifies the address through a register.
 - MOV R7, #27H
 - MOV A, @R7
(R7 holds address 27H => read the address from register R7)



DIRECT AND INDIRECT ADDRESS AREA

- The lower 128*8 addresses can be divided into 3 segments:

- Register Banks 0-3: 00H through 1FH (32 bytes). The microcontroller after reset default to register bank 0. Register bank selection is done by software. Each register bank contains 8 one-byte registers, 0 through 7.

← 8 BYTES →		
78H		7FH
70H		77H
68H		6FH
60H		67H
58H		5FH
50H		57H
48H		4FH
40H		47H
38H		3FH
30H		37H
28H	...7FH	2FH
20H	0...	27H
18H	3	1FH
10H	2	17H
08H	1	0FH
00H	0	07H

SCRATCH
PAD AREA

BIT ADDRESSABLE
SEGMENT

REGISTER
BANKS

- Bit Addressable Area: 20H through 2FH (16 bytes). Each one of the 128 bits can be directly addressed (00H through 7FH). Each of the 16 bytes can be addressed as a byte.
- Scratch Pad Area: 30H through 7FH (80 bytes). Used for data RAM.

SPECIAL FUNCTION REGISTERS

- The SFRs marked with * are both bit and byte addressable
- The other are only byte addressable.
- After reset, each SFR is loaded with a value that doesn't interfere with possible user values.

Similarity with ALP:

- $ACC = AX$
- $B = BX$
- $PSW = \text{Flag register}$

Symbol	Description	Address	Reset value
ACC*	Accumulator	E0H	00000000
B*	B Register	F0H	00000000
PSW*	Program Status Word	D0H	00000000
SP	Stack Pointer	81H	00000111
DPTR	Data Pointer 2 Bytes		
DPL	Low Byte	82H	00000000
DPH	High Byte	83H	00000000
P0*	Port 0	80H	11111111
P1*	Port 1	90H	11111111
P2*	Port 2	A0H	11111111
P3*	Port 3	B0H	11111111
IP*	Interrupt Priority Control	B8H	xxx00000
IE*	Interrupt Enable Control	A8H	0xx00000
TMOD	Timer/Counter Mode Control	89H	00000000
TCON*	Timer/Counter Control	88H	00000000
TH0	Timer/Counter 0 High Byte	8CH	00000000
TL0	Timer/Counter 0 Low Byte	8AH	00000000
TH1	Timer/Counter 1 High Byte	8DH	00000000
TL1	Timer/Counter 1 Low Byte	8BH	00000000
SCON*	Serial Control	98H	00000000
SBUF	Serial Data Buffer	99H	xxxxxxxx
PCON	Power Control	87H	00xx0000

80C51 MICROCONTROLLER FAMILY

Introduction

Memory addressing

Interrupts system

Timer system registers

Serial communication

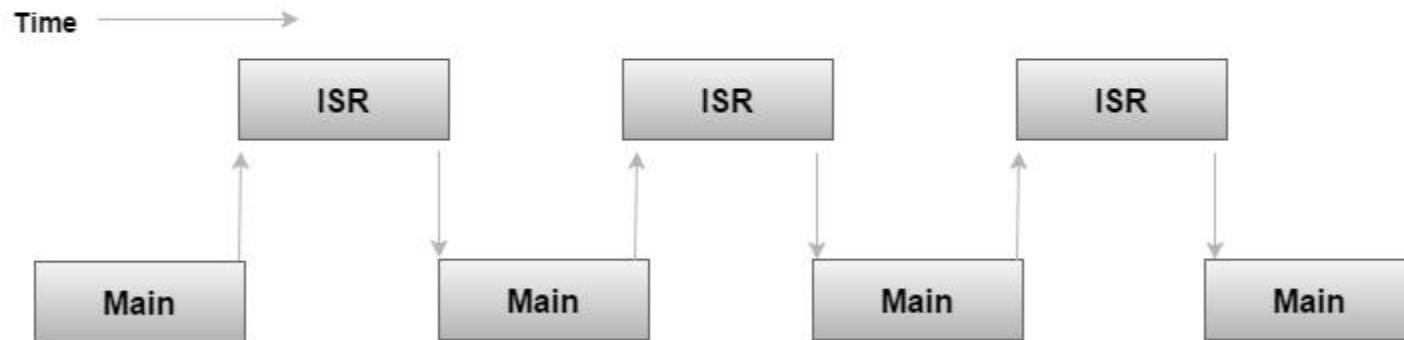
Interrupts System

- When an interrupt occurs, the microcontroller suspends temporarily the program execution and starts running the specific interrupt routine

Program Execution without Interrupts



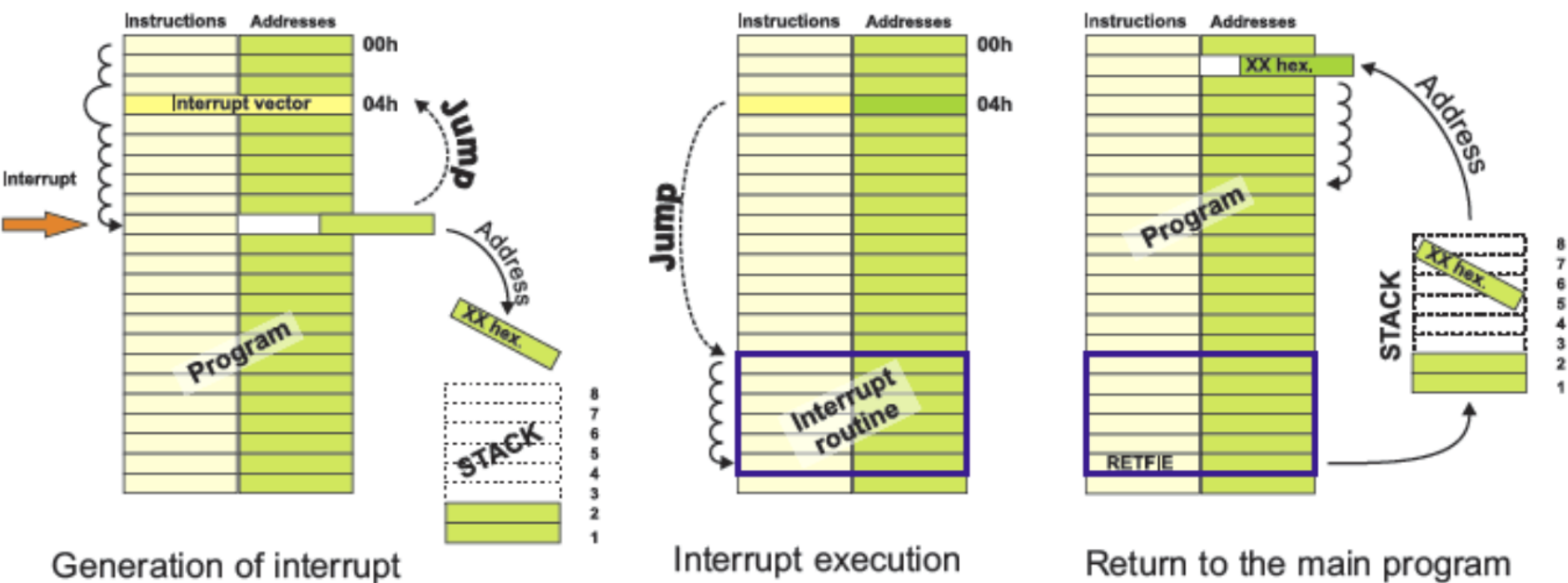
Program Execution with Interrupts



ISR : Interrupt Service Routine

Interrupts System

- When an interrupt occurs, the microcontroller suspends temporarily the program execution and starts running the specific interrupt routine
- For execution of interrupt routine the following steps are needed:
 - Save on stack the address of the next program instruction (2 octets) and of the PSW (2 octets).
 - Determine source of interrupt (interrupt number). Any interrupt source has an interrupt vector, keeping the jump instruction at the address when starts the interrupt routine. Vector address is calculated by microcontroller. Interrupt vectors (8 octets) are stored into a specific table.
 - Accesses the interrupt vector table and finds the routine address.
 - Executes the interrupt routine.
 - Interrupt routine ends with IRET instruction, loading from stack the location of the next program instruction (2 octets) and PSW (2 octets)



- Interrupts implementation :
 - Bit EA from register **IE** set to 1; means **interrupts enabling**
 - Set to 1 all bits in register IE for interrupts will be used
 - Each interrupt vector will be loaded with a long jump instruction to the address of the interrupt routine
 - For **external interrupts**, pins INT0\ (P3.2) and INT1\ (P3.3) must be set to 1, and corresponding bits (IT0 și IT1) from TCON register must be set or reset for activating interrupts on level or edge basis.
- 80C51 microcontroller interrupts

INTERRUPT SOURCE	DESCRIPTION	VECTOR ADDRESS
IE0	External interrupt 0	0003H
TF0	Timer 0 overflow	000BH
IE1	External interrupt 1	0013H
TF1	Timer 1 overflow	001BH
RI&TI	Serial interrupts	0023H
TF2&EXF2	Timer 2 overflow or external interrupts	002BH

IE Register structure

- Bit or byte addressable. If EA is 0, all interrupts are disabled. If EA is 1, any interrupt is enabled by setting its bit to 1.
- Bit functions:
 - EA, IE.7: if EA=0, neither interrupt will be acknowledged. If EA=1, each interrupt source may be individually enabled or disabled.
 - ET2, IE.5: timer 2 overflow
 - ES, IE.4: interrupt serial port
 - ET1, IE.3: Timer 1 overflow
 - EX1, IE.2: external interrupt 1
 - ET0, IE.1: Timer 0 overflow
 - EX0, IE.0: external interrupt 0

EA	-	ET2	ES	ET1	EX1	ET0	EX0
----	---	-----	----	-----	-----	-----	-----

Interrupts priorities

- Two priority levels. The higher one may interrupt the other. For priority assignement corresponding bit from IP register must be set to 1 or 0.
- Priorities from high to low are: IE0, TF0, IE1, TF1, RI or TI and TF2 or EXF2.

IP register (Interrupt Priority)

- Bit or byte adressable.
- Bit functions:
 - PT2, IP.5: Timer 2 interrupt priority level (only 8052)
 - PS, IP.4: serial port priority level
 - PT1, IP.3: Timer 1 interrupt priority level
 - PX1,IP.2: external interrupt 1 priority level
 - PT0, IP.1: Timer 0 interrupt priority level
 - PX0, IP.0: external interrupt 0 priority level

-	-	PT2	PS	PT1	PX1	PT0	PX0
---	---	-----	----	-----	-----	-----	-----

80C51 MICROCONTROLLER FAMILY

Introduction

Memory addressing

Interrupts system

Timer system registers

Serial communication

Timer system registers

- 8051 has two timers, Timer 0 and Timer 1 and associated registers (TCON and TMOD)
 - They can be used as timers or as event counters.

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

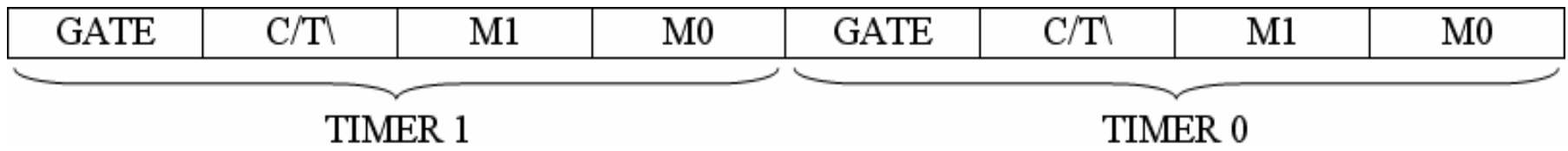
TCON register (Timer Control): used to start or stop the Timers of 8051

- Bit or byte addressable. Bit functions:
 - TF1, TCON.7: Flag Timer 1 overflow. Hardware erased when interrupt routine is running
 - TR1, TCON.6: Timer 1 control bit. if TR1=1, Timer 1 ON. if TR1=0, Timer 1 OFF.
 - TF0, TCON.5: Flag Timer 0 overflow. Same as Flag Timer 1 overflow.
 - TR0, TCON.4: Timer 0 control bit. Same as for Timer 1.
 - IE1, TCON.3: Flag for external interrupt 1. Set hardware when external interrupt 1 edge detected; hardware reset when processing interrupt.
 - IT1, TCON.2: Interrupt 1 type control bit. If IT1=1, interrupt 1 is triggered by a falling down edge. If IT1=0, interrupt 1 is triggered by level 0.
 - IE0, TCON.1: Flag for external interrupt 0. Set hardware when interrupt edge detected, hardware erased when interrupt processing.
 - IT0, TCON.0: Interrupt 0 type control bit . If IT0=1, interrupt 0 is triggered by falling edge. If IT0=0, interrupt 0 is triggered by logic level 0.

TMOD Register (Timer Mode): used to set the Operating Modes of the Timers T0 and T1

- Byte addressable. Bit functiones:
- GATE: If GATE=1, TIMERx will run if TRx=1 and INTx=1 (hardware control). If GATE=0, TIMERx will run as TRx=1 (software control).
- C/T\, Timer or Counter Selector. If C/T\=0, Timer operation (input from internal clock). If C/T\=1, Counter operation (input from Tx).
- M1, M0: operation mode selection

M1	M2	Mode
0	0	13-bit timer mode.
0	1	16-bit timer mode.
1	0	8-bit auto reload mode.
1	1	Spilt mode.



80C51 MICROCONTROLLER FAMILY

Introduction

Memory addressing

Interrupts system

Timer system registers

Serial communication

Serial Communication

SCON Register (Serial Control): used to control the Serial Port

- Bit or byte addressable. Bit functions:
 - SM0: Bit 0 Serial Port mode (MSB).
 - SM1: Bit 1 Serial Port mode(LSB).
 - SM2: Activates communications modes
 - REN: Software set or reset for Reception enable/disable
 - TB8, RB8: Special role in operation modes 2 and 3
 - TI: Transmission interrupt
 - RI: Reception interrupt

SM1 SM0		Mode	Operating Mode
0	0	0	Shift register
0	1	1	8-bit UART
1	0	2	9-bit UART
1	1	3	9-bit UART

SM0	SM1	SM2	REN	TB8	RB8	TI	RI
-----	-----	-----	-----	-----	-----	----	----

mode 0: port communicates with fixed baud rate

- Baud rate = $\text{Osc Freq}/12$
- Port acts as a 8bit shift register; data sent with LSB first through RXD pin.

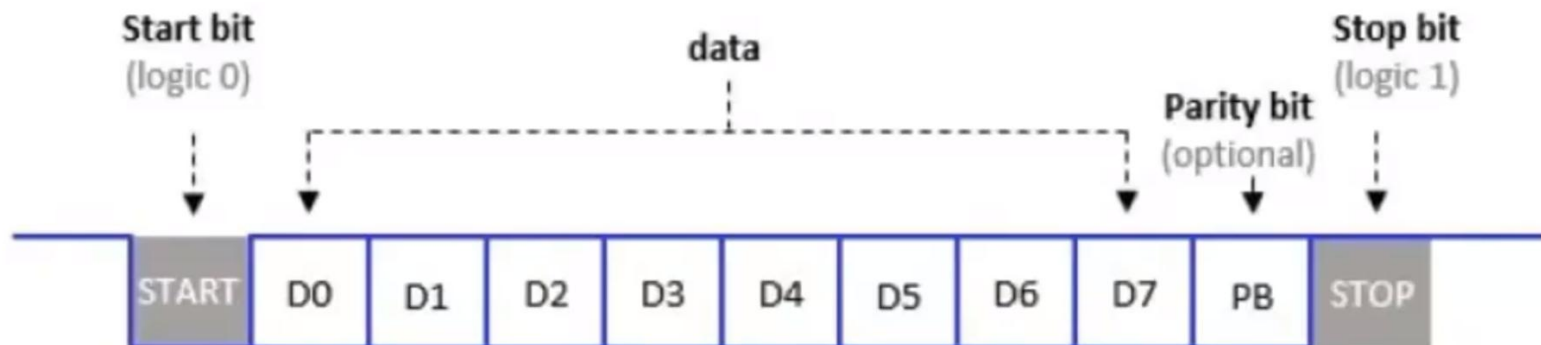
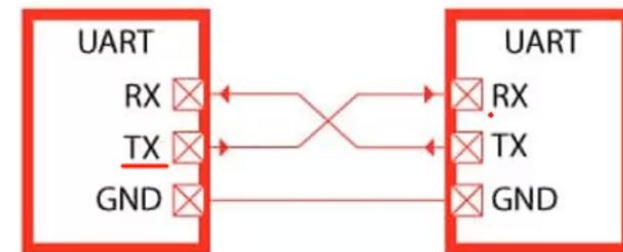
mode 1: variable baud rate, generated by Timer 1

- Baud rate = $(K * \text{Osc Freq}) / \{32 * 12 * [256 - (\text{TH1})]\}$
- Bit SMOD from PCON register determines K value. If SMOD = 0, K = 1, if SMOD = 1, K = 2.
- $\text{TH1} = 256 - (K * \text{Osc Freq}) / (384 * \text{Baud Rate})$
- Frame format: 1 start bit, 8 data bit, 1 stop bit.
- Interrupts TI și RI are activated when a frame was sent or received

Synchronous communication (CLK)

vs

Asynchronous communication (no CLK)

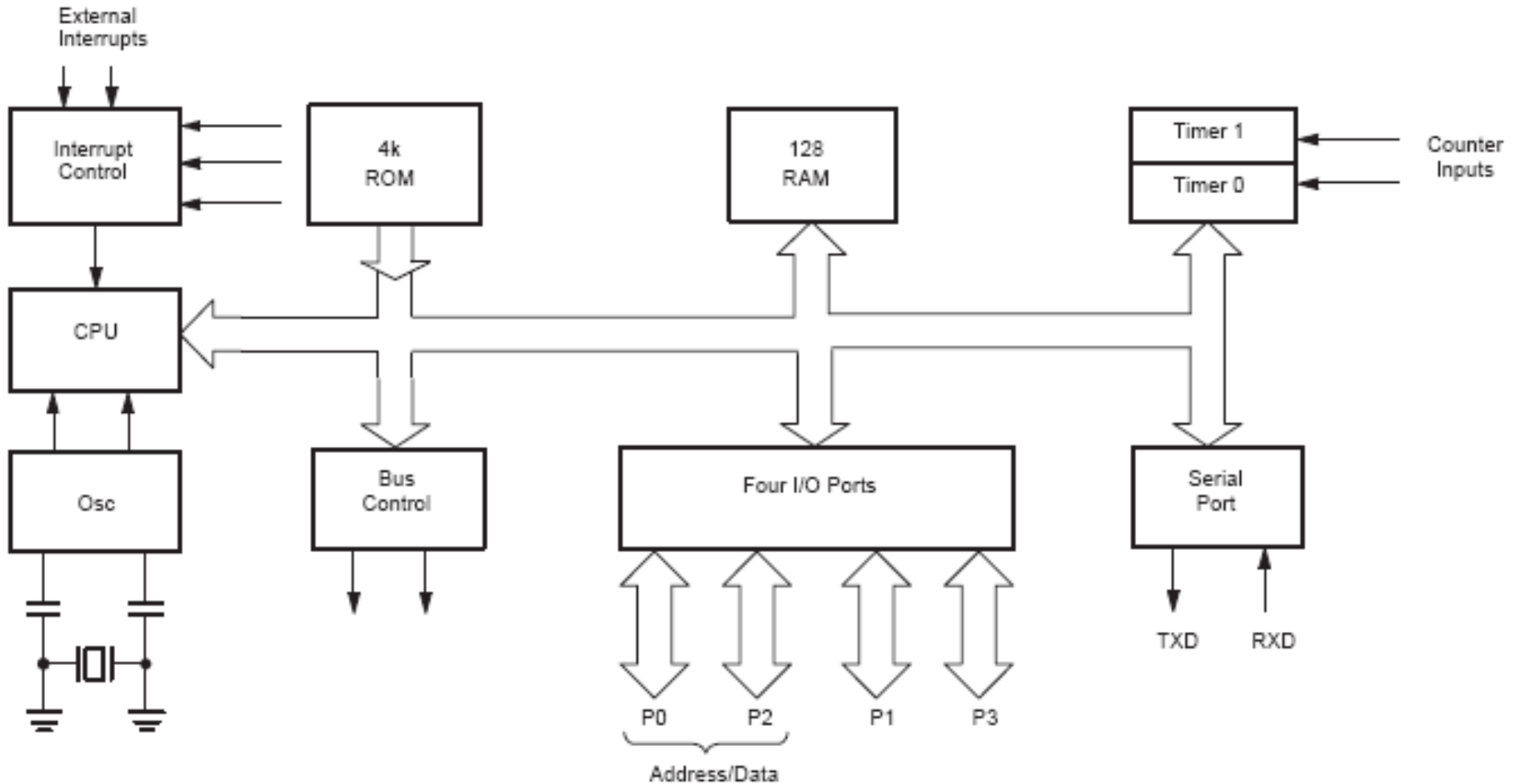


mode 2: fixed baud rate communication

- $\text{SMOD} = 1$, Baud rate = $1/32 \times \text{Osc Freq}$.
- $\text{SMOD} = 0$, Baud rate = $1/64 \times \text{Osc Freq}$.
- Frame format: 1 start bit, 8 data bit, 1(stick) bit, 1 stop bit. The 9th bit (stick bit) is used for interprocessor communications

mode 3: similar to mode 2 as frame format and the role of stick bit, and as baud rates is similar to mode 1

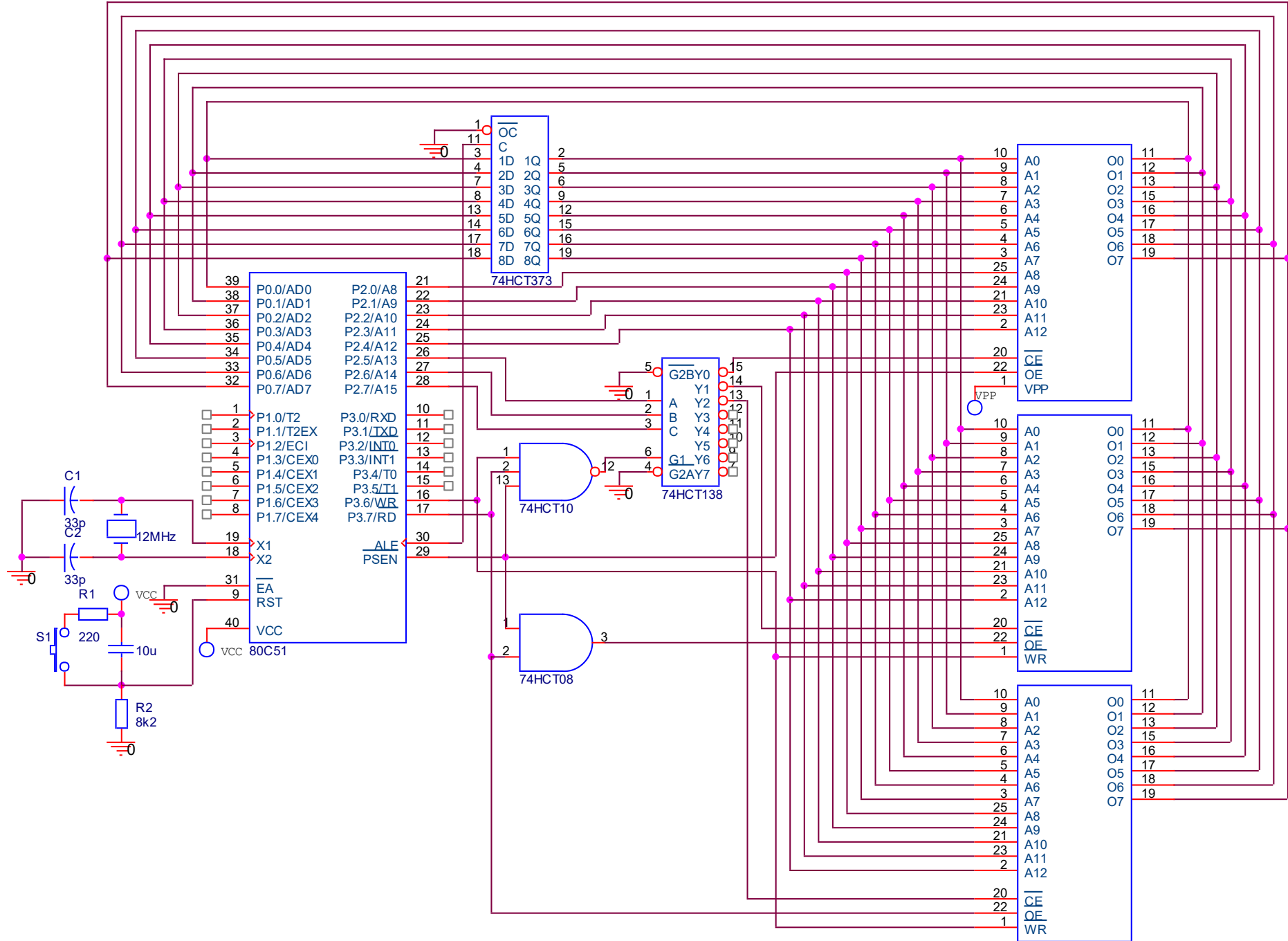
Block diagram – *remember it*



Proposed problems

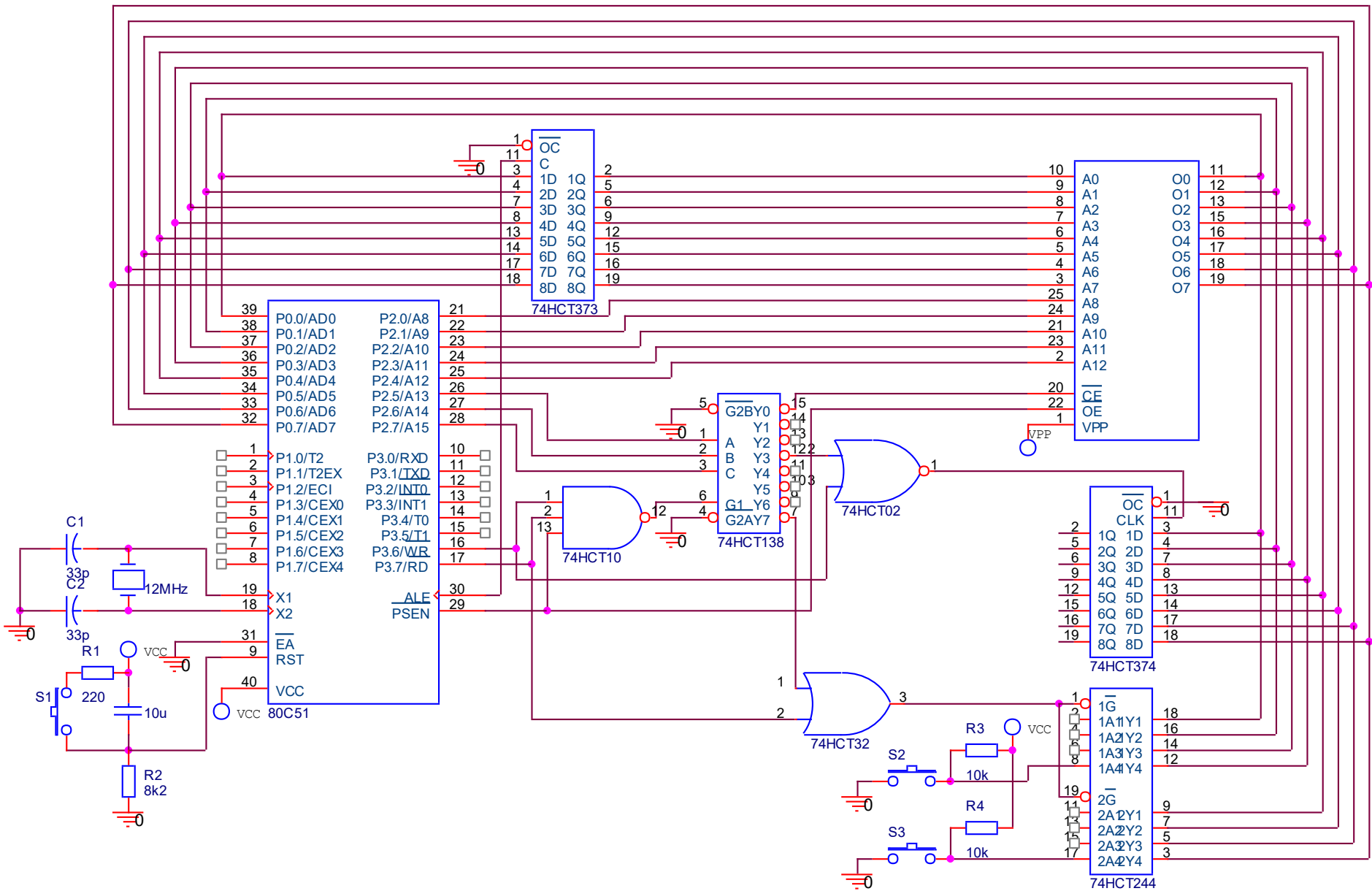
Design a system based on a 80C51 microcontroller system, with the following blocks:

- 12MHz quartz oscillator
- Reset circuit
- 8k * 8 external program memory with the low (base) address 0000_H
- 8k * 8 external data and program memory with the low (base) address 2000_H
- 8k * 8 external data memory with the low (base) address 4000_H



Design a system based on an 80C51 microcontroller system, with the following blocks:

- 12MHz quartz oscillator
- Reset circuit
- 8k * 8 external program memory with the low (base) address 0000H
- Output port at address 011XXXXXXXXXXXXXX_b
- Input port at address 111XXXXXXXXXXXXXX_b to which 2 keys are connected

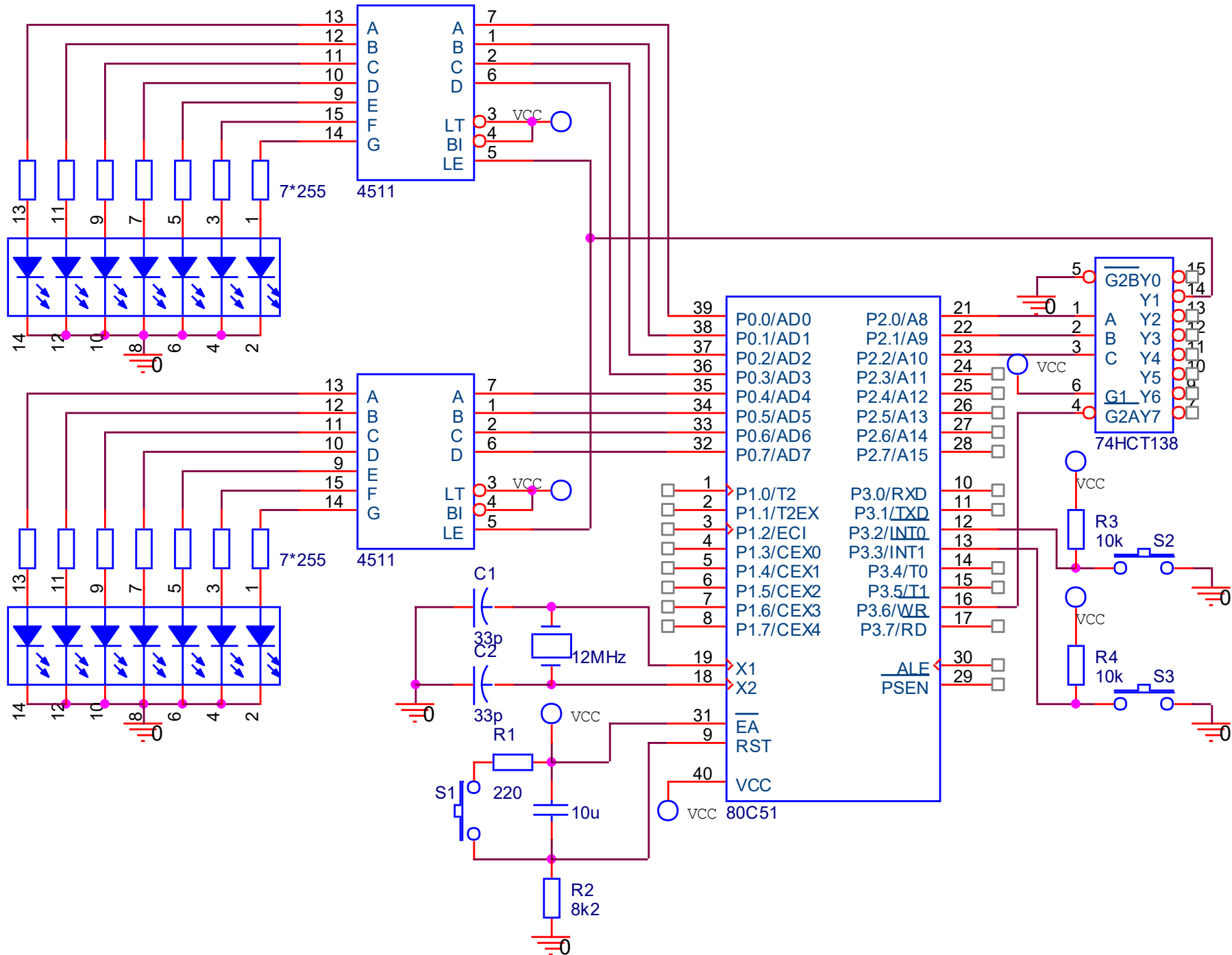


Design a system based on an 80C51 microcontroller system, with the following blocks:

- 12MHz quartz oscillator
- Reset circuit
- Output port at address $XXXXX001XXXXXXX_b$ having connected two 7-segments LED displays using common cathode, with: $V_{LED}=1,7V$ and $I_{LED}=10mA$
- 2 keys connected at interrupt input lines
- Only internal program memory is used

Calculus for resistor values (for the LED based display) is given by the formula:

$$R=(V_{OHTYP}-V_{LED})/I_{LED}=(4,25V-1,7V)/10mA=255\Omega$$

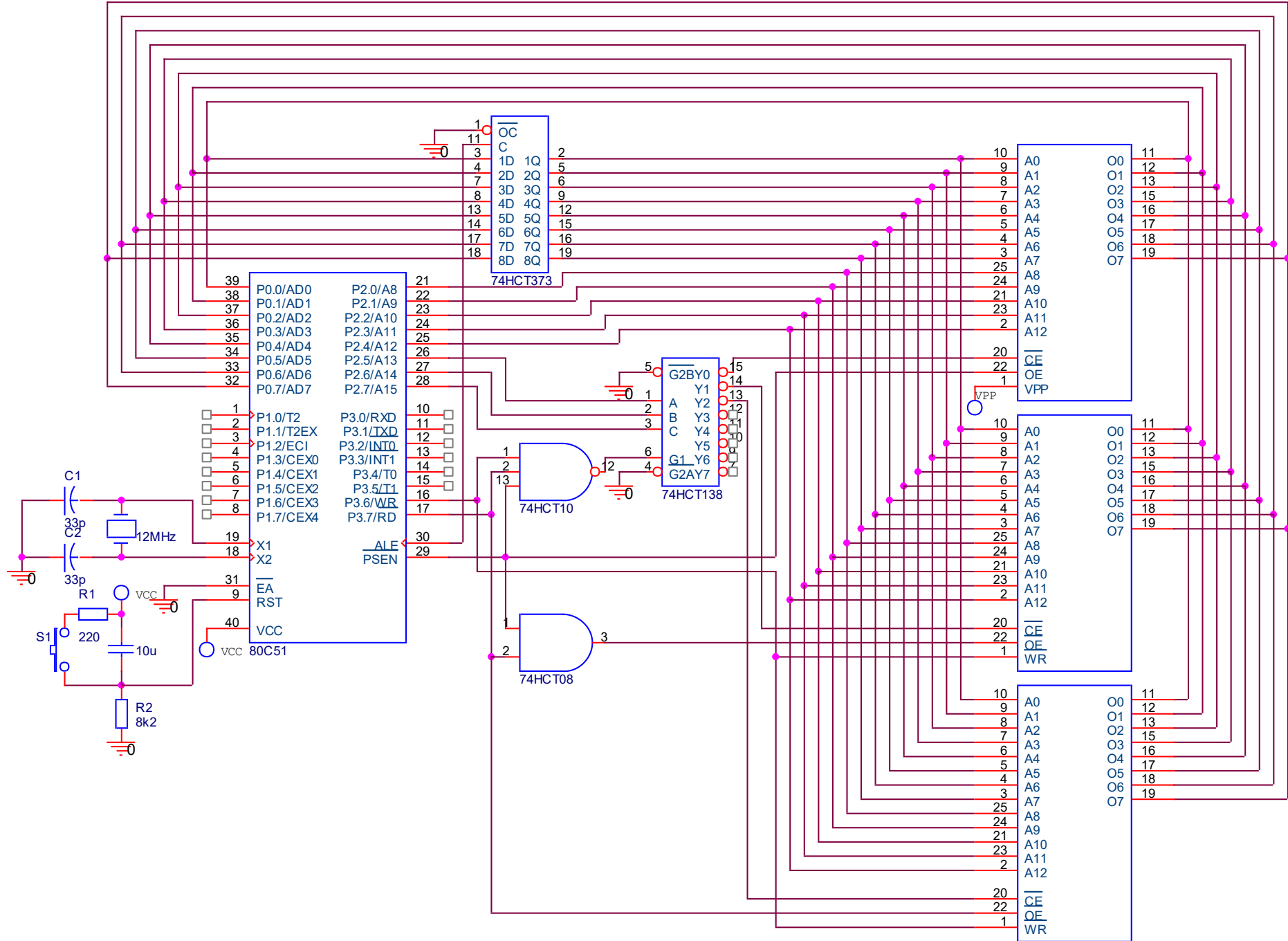


Solutions explained

Proposed problems

Design a system based on a 80C51 microcontroller system, with the following blocks:

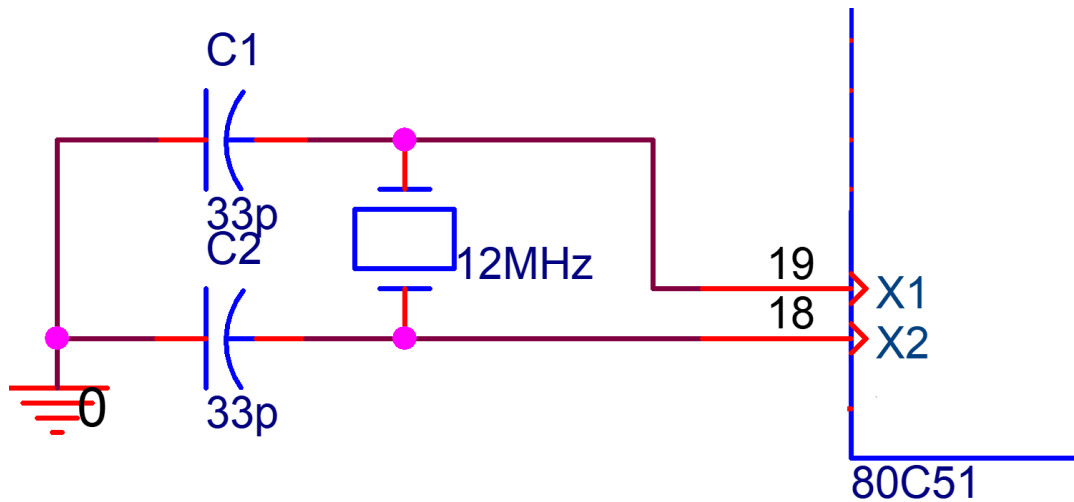
- 12MHz quartz oscillator
- Reset circuit
- 8k * 8 external program memory with the low (base) address 0000_H
- 8k * 8 external data and program memory with the low (base) address 2000_H
- 8k * 8 external data memory with the low (base) address 4000_H



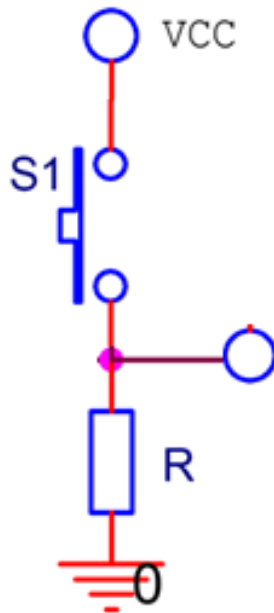
Proposed problems

Design a system based on a 80C51 microcontroller system, with the following blocks:

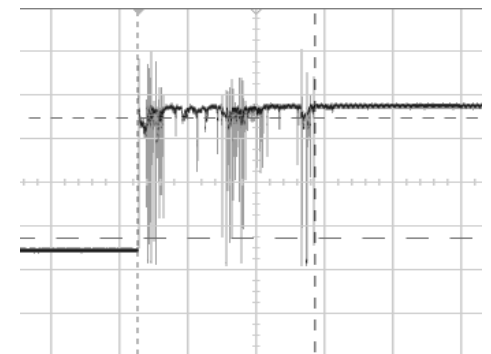
- 12MHz quartz oscillator
- Reset circuit
- 8k * 8 external program memory with the low (base) address 0000H
- 8k * 8 external data and program memory with the low (base) address 2000H
- 8k * 8 external data memory with the low (base) address 4000H

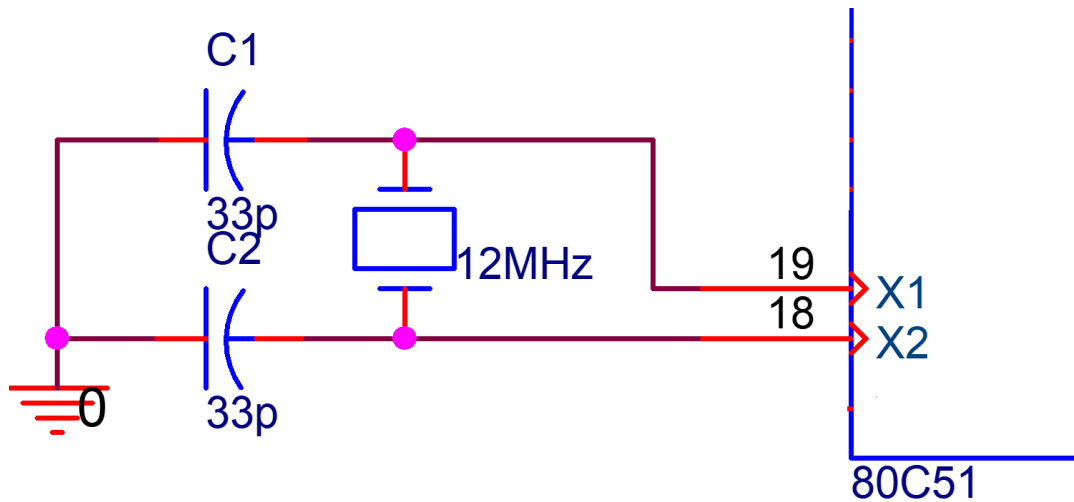


- 12MHz quartz oscillator

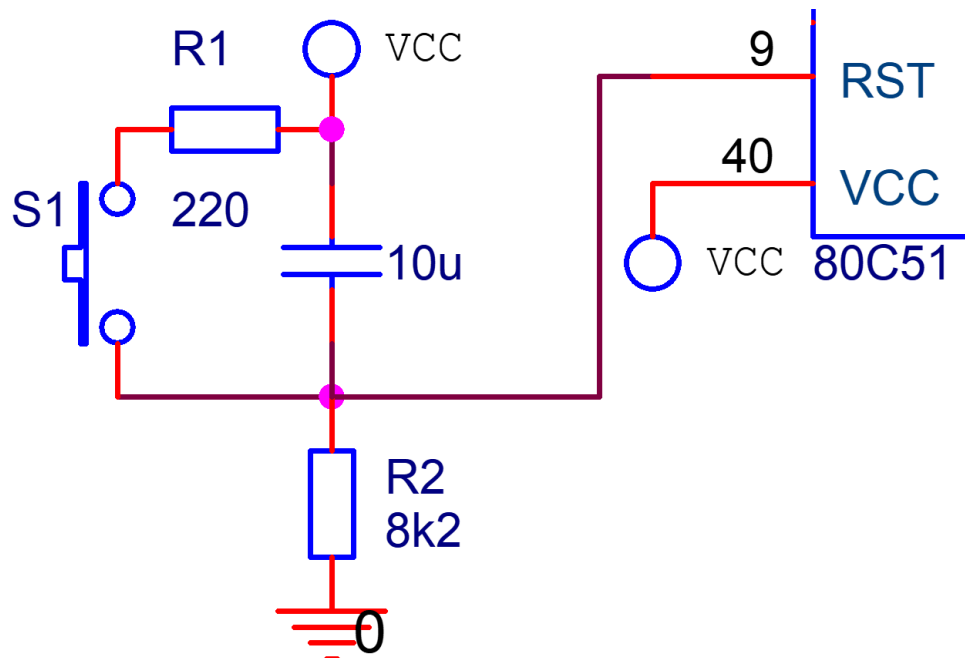


- Reset circuit
 - activated (set to logic 1 – Vcc) for 2 two machine cycles
 - NOISE





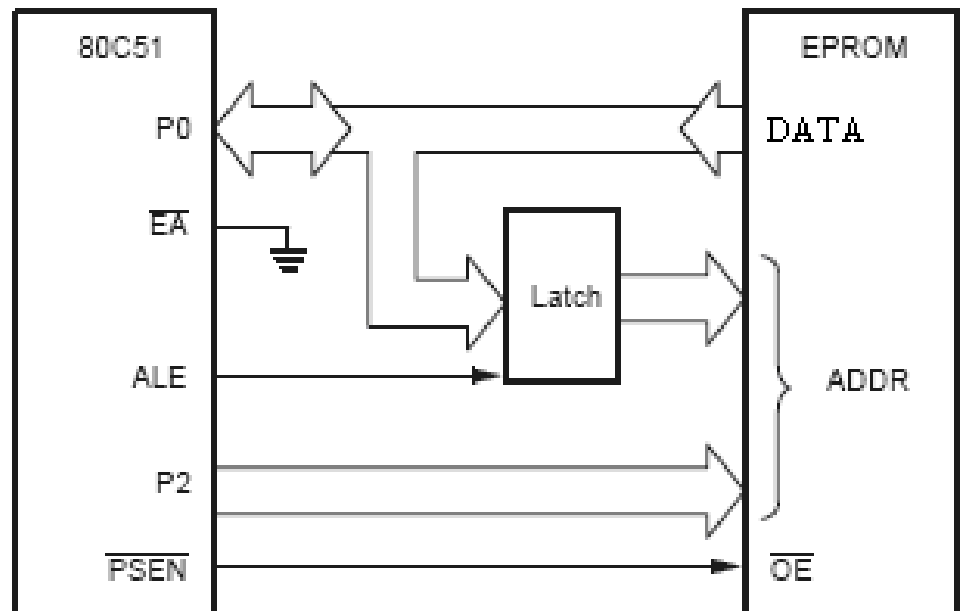
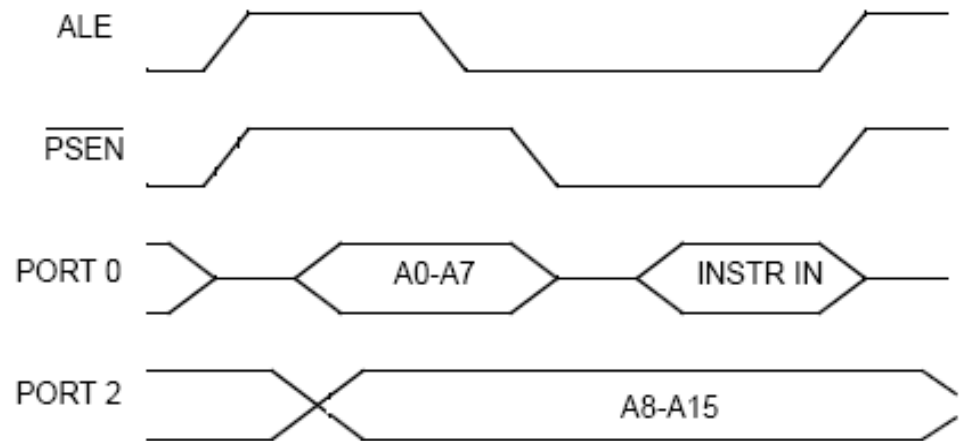
- 12MHz quartz oscillator

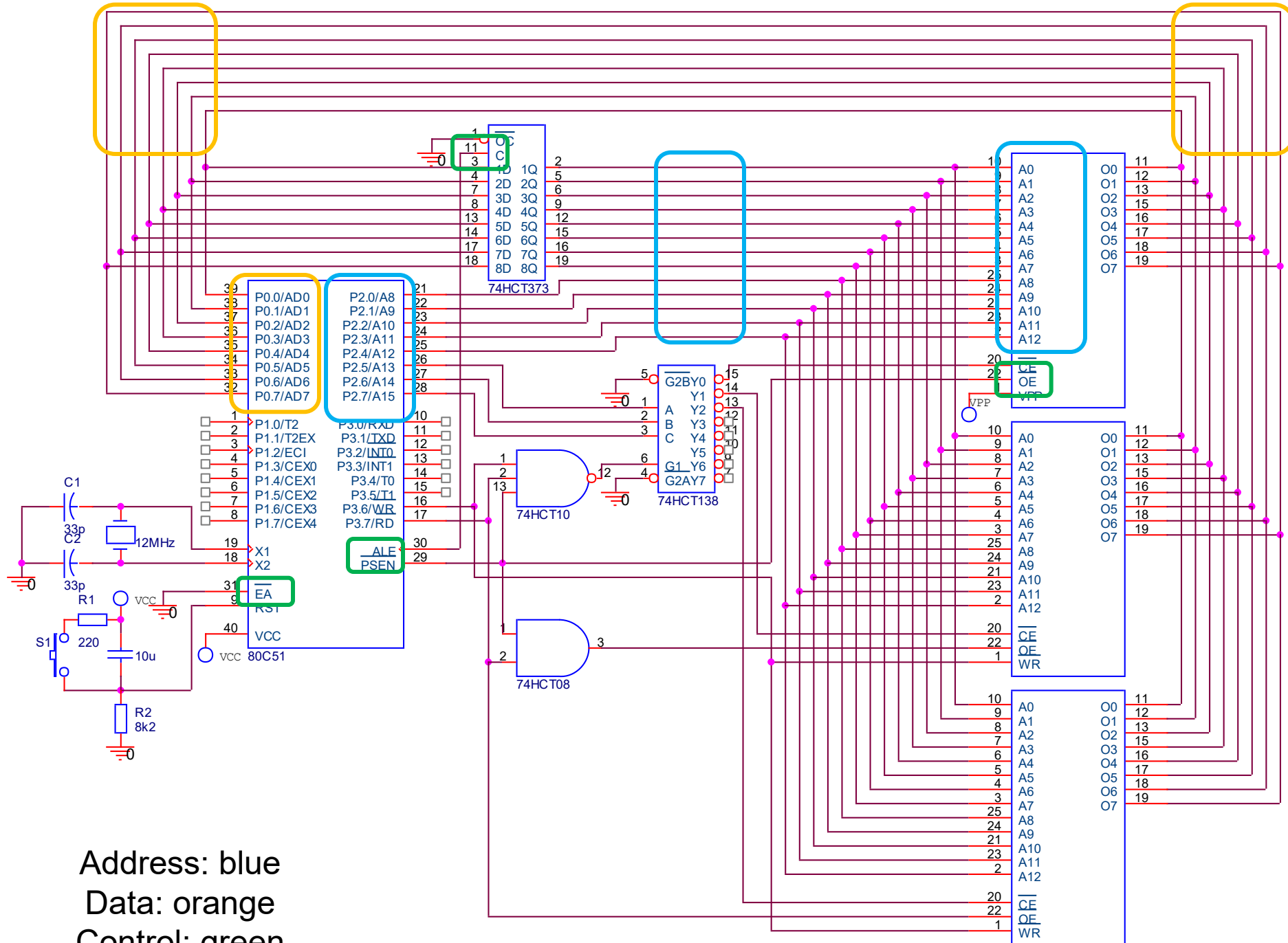


- Reset circuit
 - activated (set to logic 1 – Vcc) for 2 two machine cycles
 - filter with capacitor to avoid noise

- **8k * 8 external program memory** with the low (base) address 0000H
- Address bus: P0 and P2
- Data bus: P0
- Control bus: ALE, PSEN, EA
- $2^{13} = 8192$ (8K)

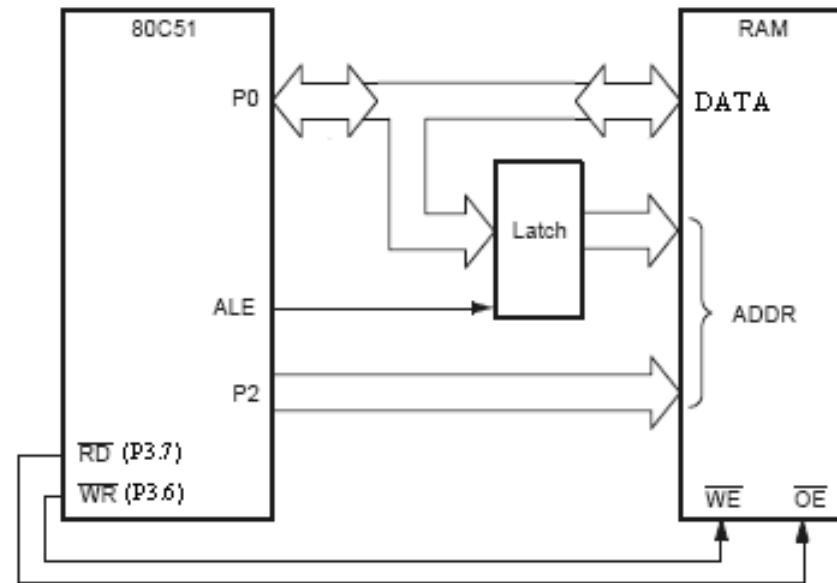
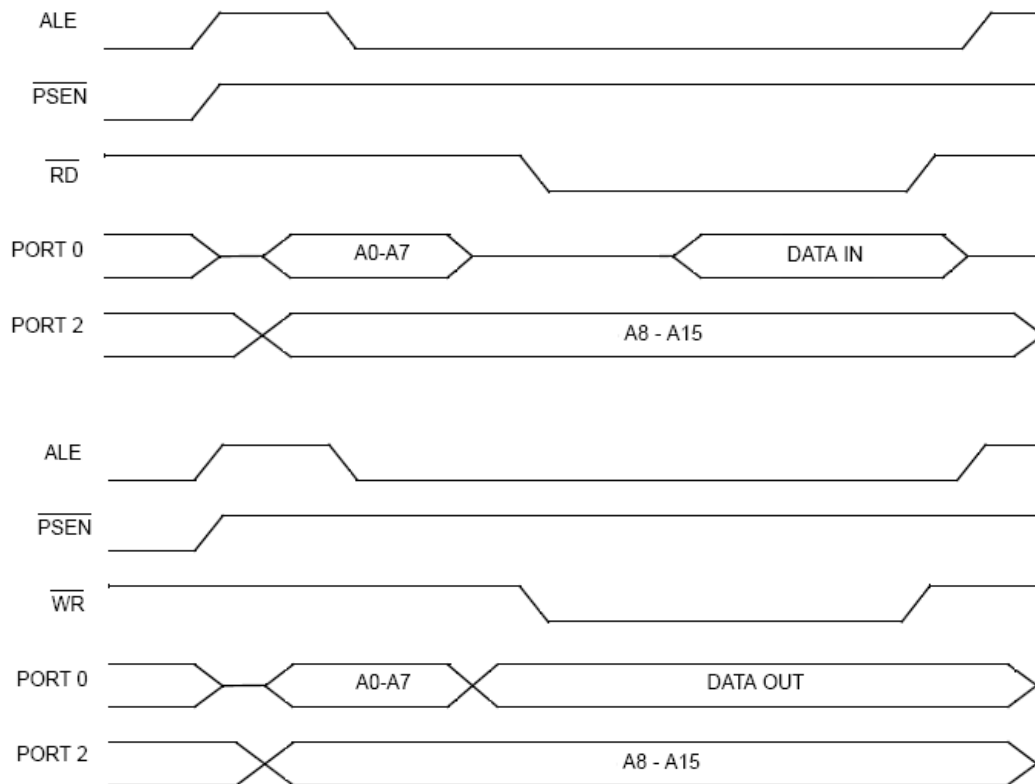
External program memory read cycle and configuration

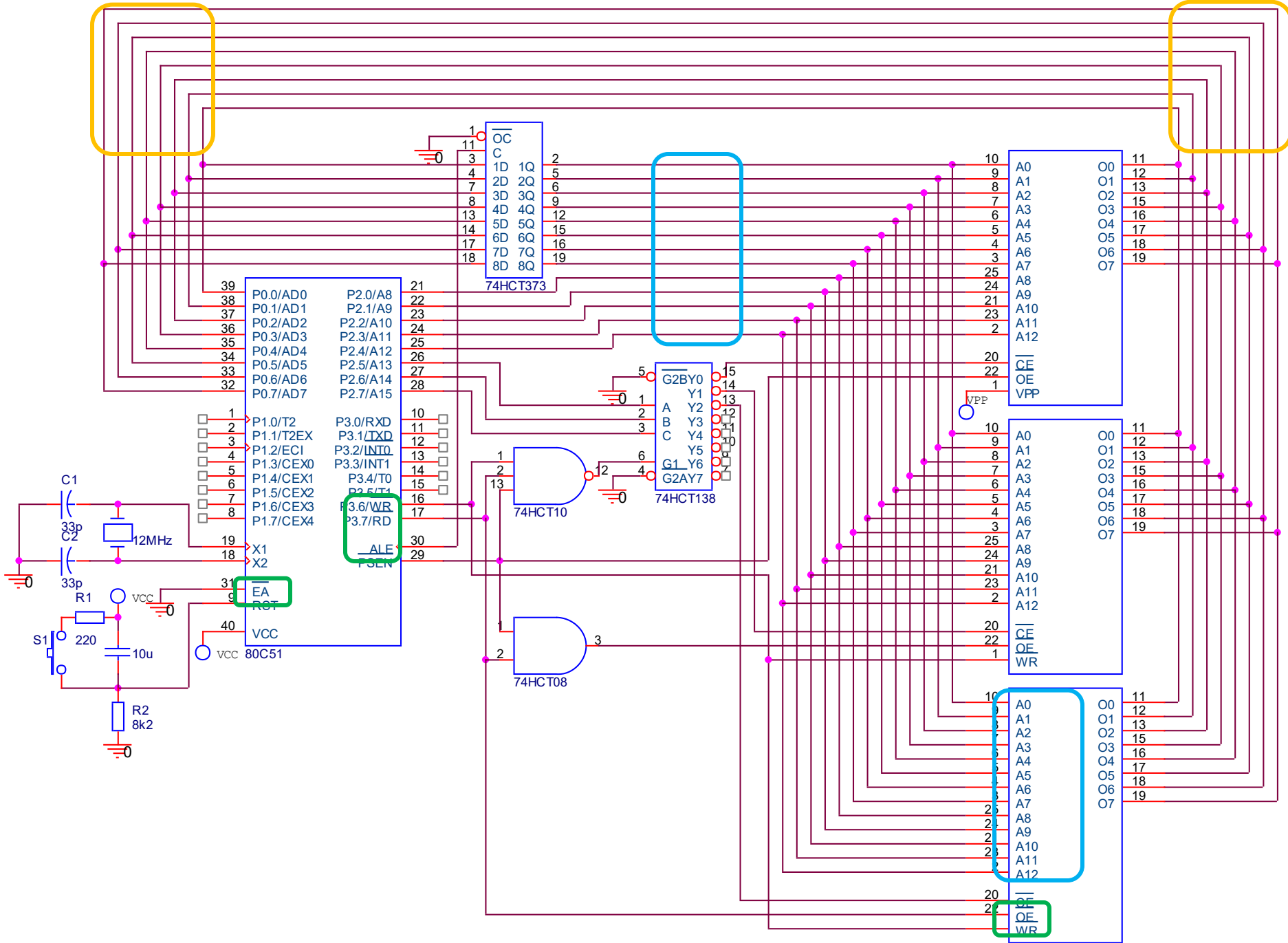




- **8k * 8 external data memory** with the low (base) address 4000H

External data memory read and write cycles





- **8k * 8 external data and program memory** with the low (base) address 2000H

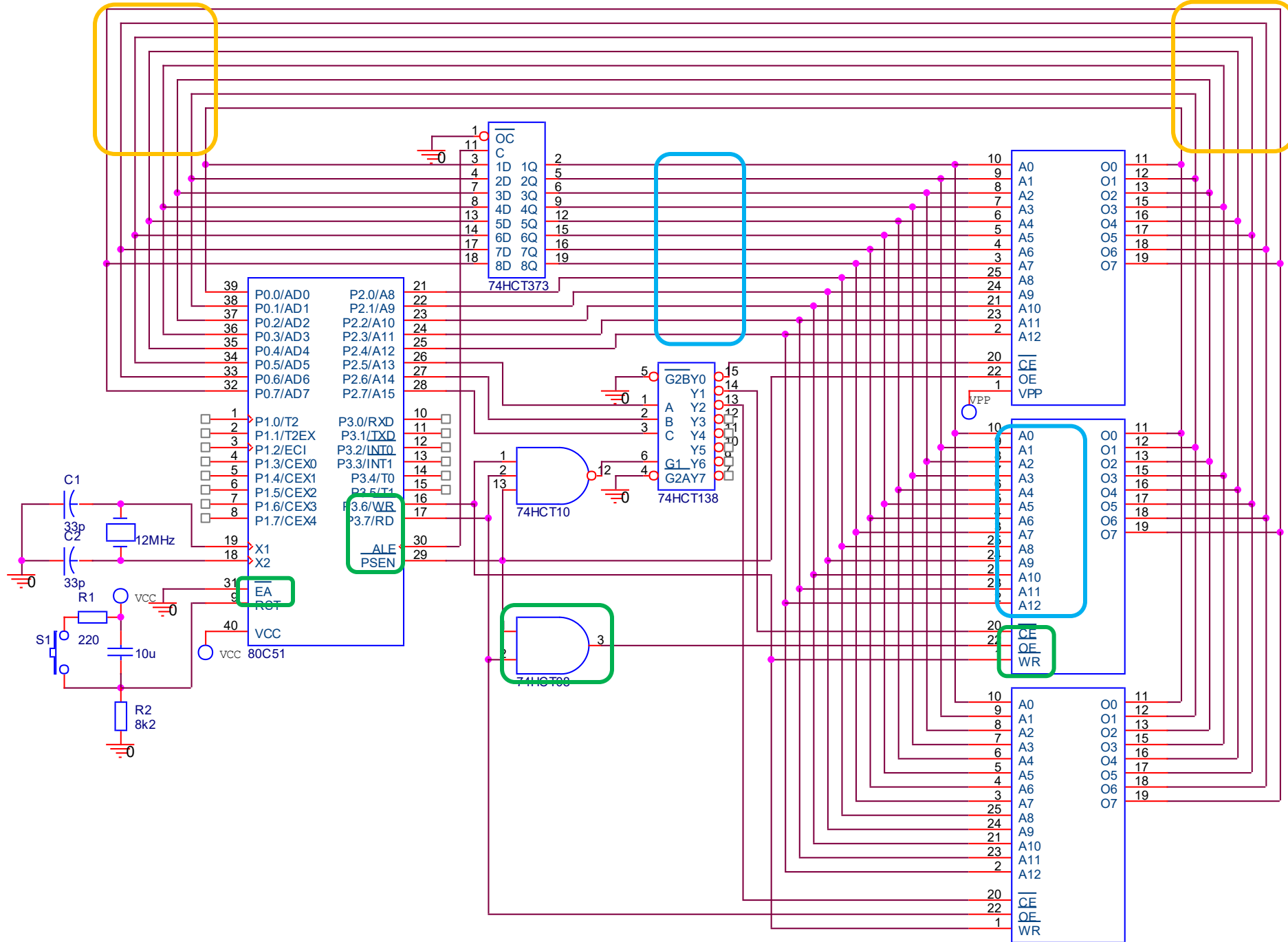
Read: program/data memory (PSEN\ or RD\ signals)

PSEN\ RD\ | **OE** (AND function)

0	0		0
0	1		0
1	0		0
1	1		1

RD and PSEN, when one is selected – active on 0, OE\ should be selected – active also on 0

Write: data memory (WR\ signals)



- 8k * 8 external program memory with the low (base) address 0000H
- 8k * 8 data and external program memory with the low (base) address 2000H
- 8k * 8 external data memory with the low (base) address 4000H

0000H: **0000**.....0b
 2000H: **0010**.....0b
 4000H: **0100**.....0b

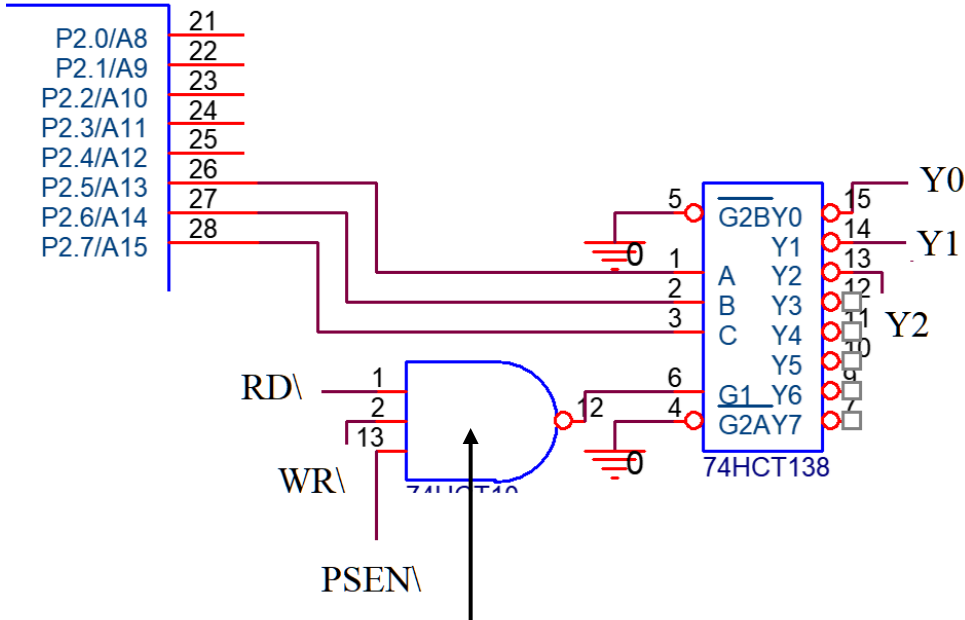
A15 A14 A13 | A12 A11... A0
 Block selection | 8k*8 address lines (13 lines)

64k*8	A15	A14	A13
8k*8	0	0	0
8k*8	0	0	1
8k*8	0	1	0
8k*8	0	1	1
8k*8	1	0	0
8k*8	1	0	1
8k*8	1	1	0
8k*8	1	1	1

← 1st mem. block: address from 0000H to 1FFFFH
 ← 2nd mem. block: address from 2000H to 3FFFFH
 ...

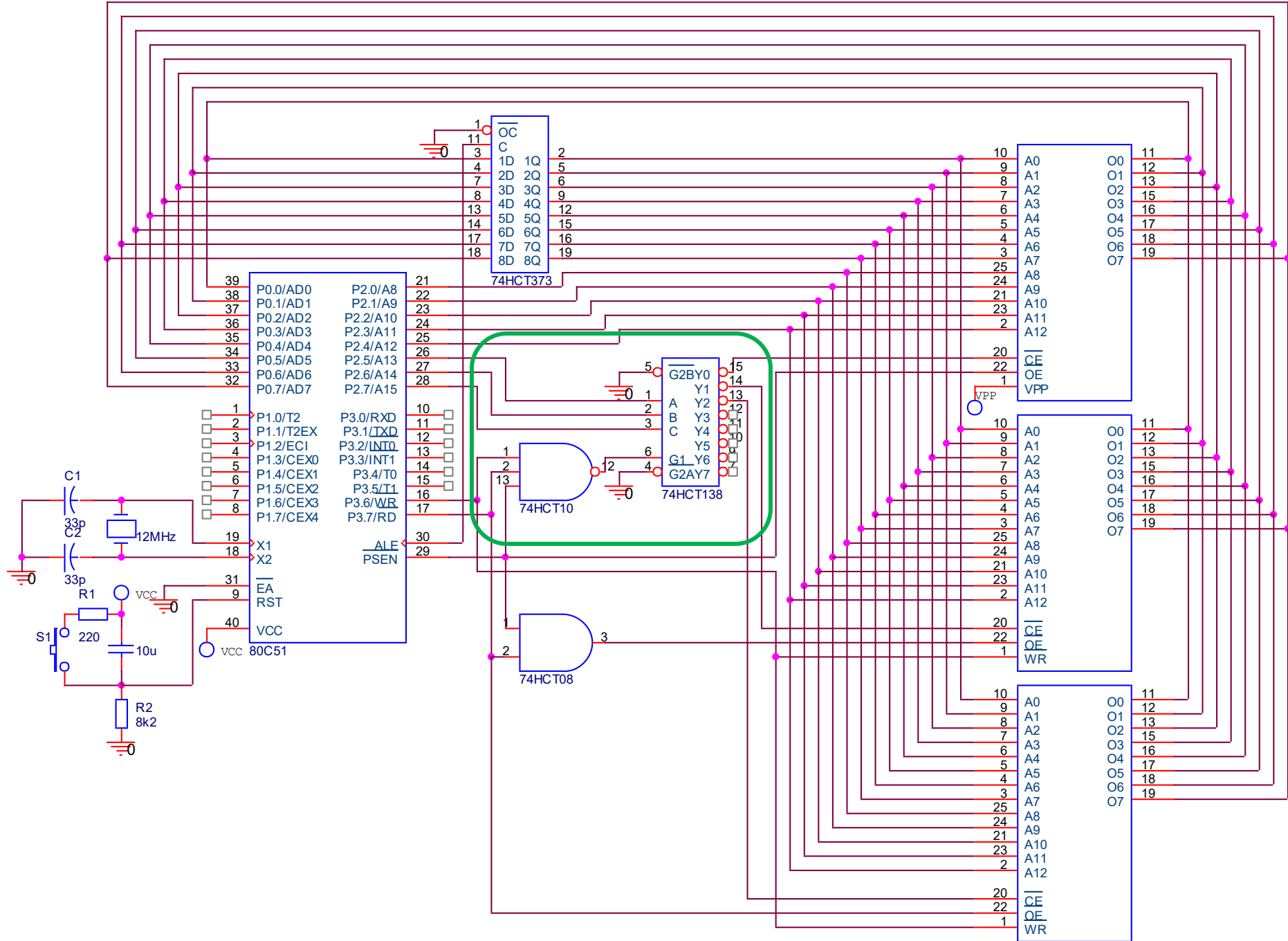
- 8k * 8 external program memory with the low (base) address 0000H
- 8k * 8 data and external program memory with the low (base) address 2000H
- 8k * 8 external data memory with the low (base) address 4000H

64k*8	A15	A14	A13
8k*8	0	0	0
8k*8	0	0	1
8k*8	0	1	0
8k*8	0	1	1
8k*8	1	0	0
8k*8	1	0	1
8k*8	1	1	0
8k*8	1	1	1



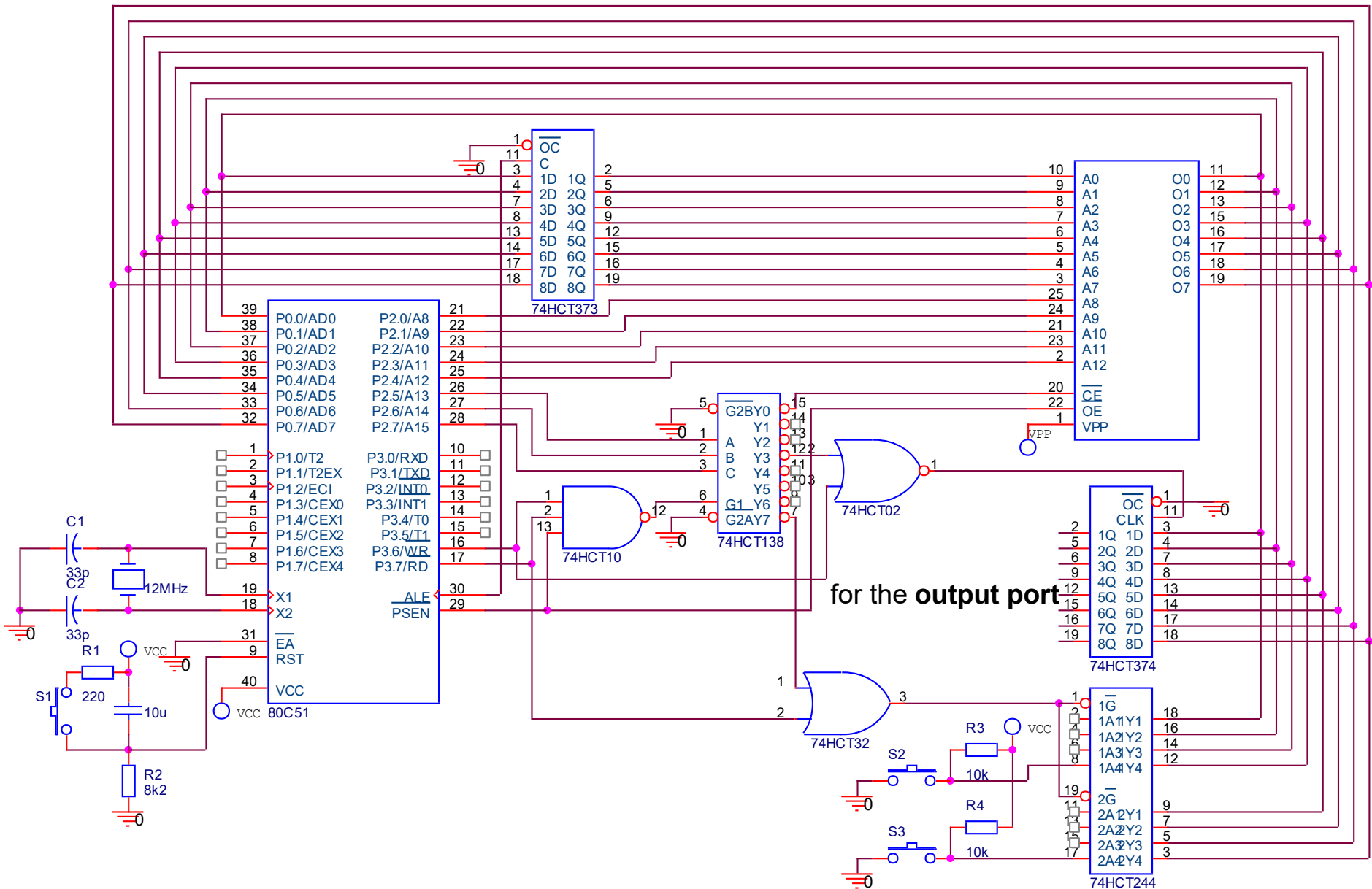
DCD is activated when either RD\, WR\ or PSEN\ are activated

RD\	WR\	PSEN\	NAND
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



Design a system based on an 80C51 microcontroller system, with the following blocks:

- 12MHz quartz oscillator
- Reset circuit
- 8k * 8 external program memory with the low (base) address 0000H
- Output port at address 011XXXXXXXXXXXXX_b
- Input port at address 111XXXXXXXXXXXXX_b to which 2 keys are connected



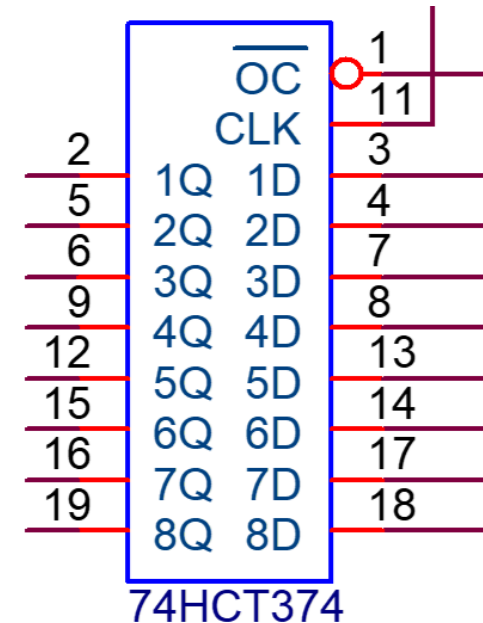
for the output port

for the input port

remember the functioning from lecture 6:

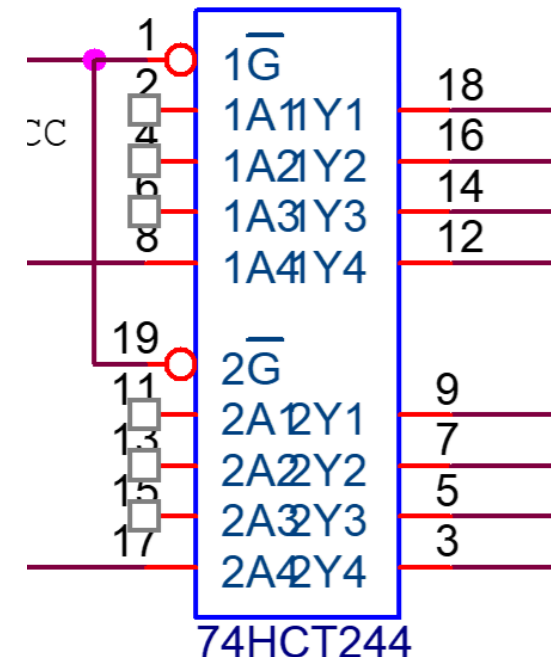
A. 74374 register (with 3ST outputs) for the **output port** (with WR\):

- Writing data into register is made on the positive front of clock signal CP
 If OE\ == 0 – transmit to the output;
 else
 “say nothing” (HZ)



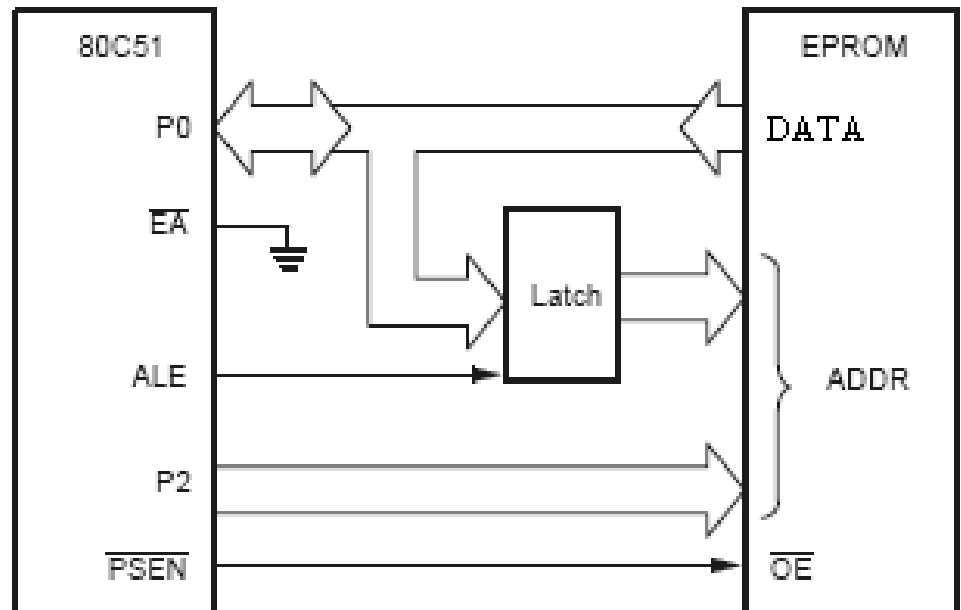
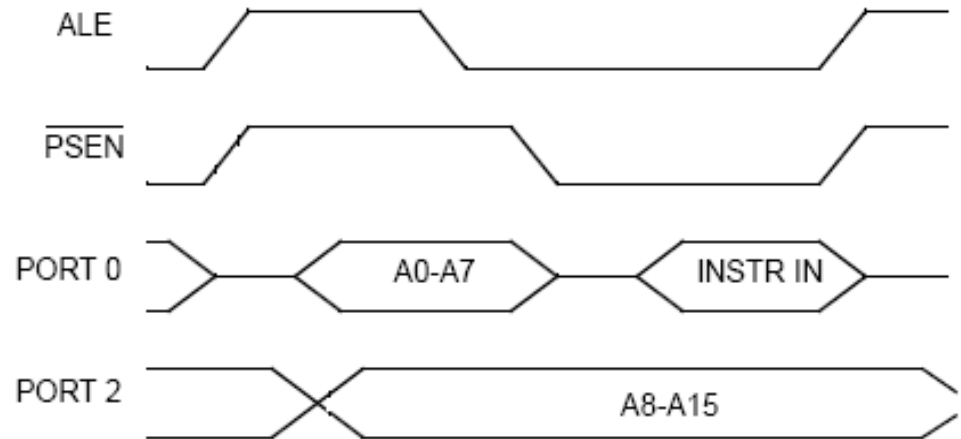
B. 74244 Unidirectional Amplifier (Digital amplifier) for the **input port** (with RD\):

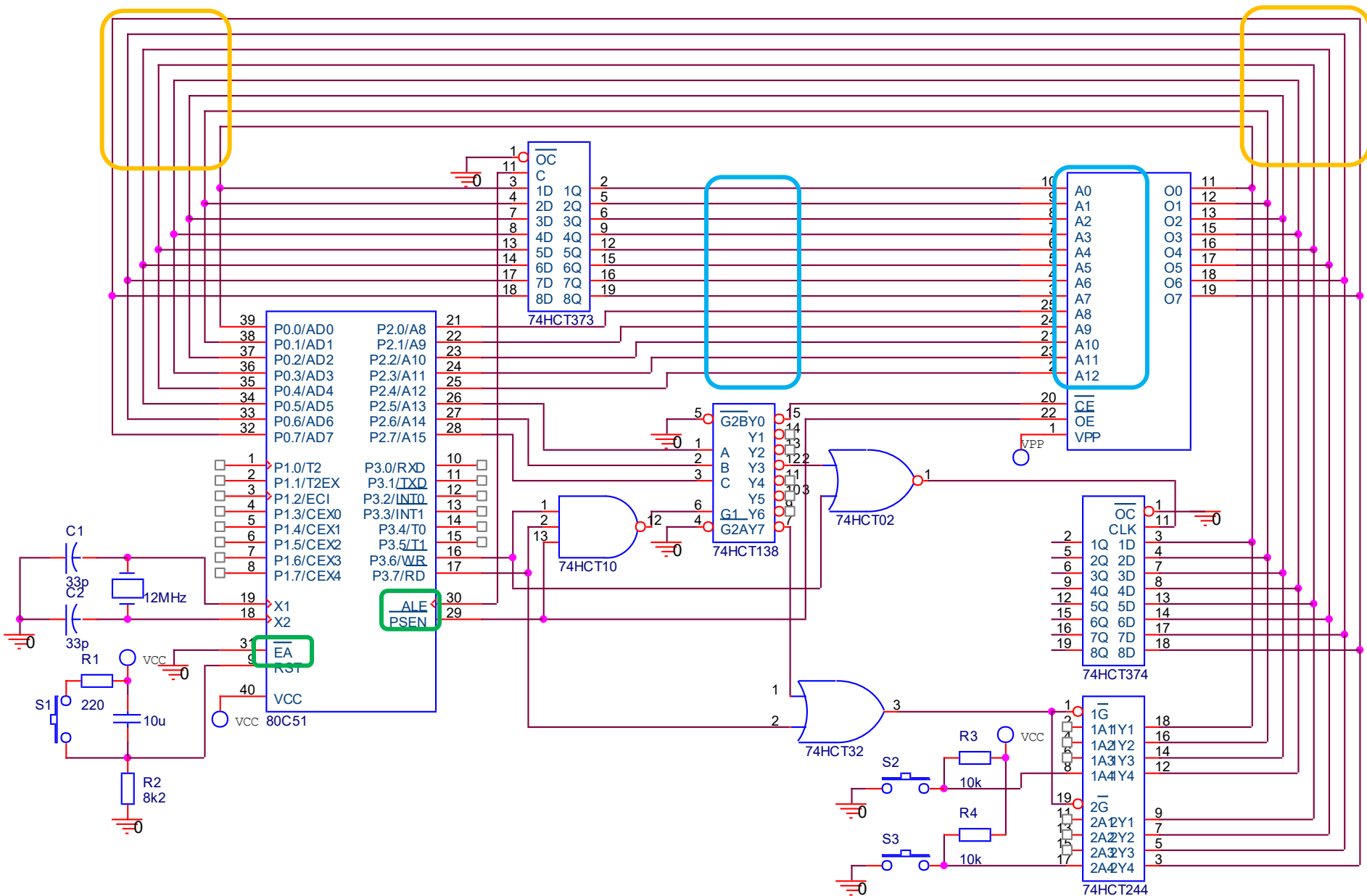
- If In=0 => Out=0
 If In=1 => Out=1
- Outputs $1Y_0 - 1Y_3$ will be enabled by low active signal $1OE\$
- Outputs $2Y_0 - 2Y_3$ will be enabled by signal $2OE\$



- **8k * 8 external program memory** with the low (base) address 0000H
- Address bus: P0 and P2
- Data bus: P0
- Control bus: ALE, PSEN, EA

External program memory read cycle and configuration



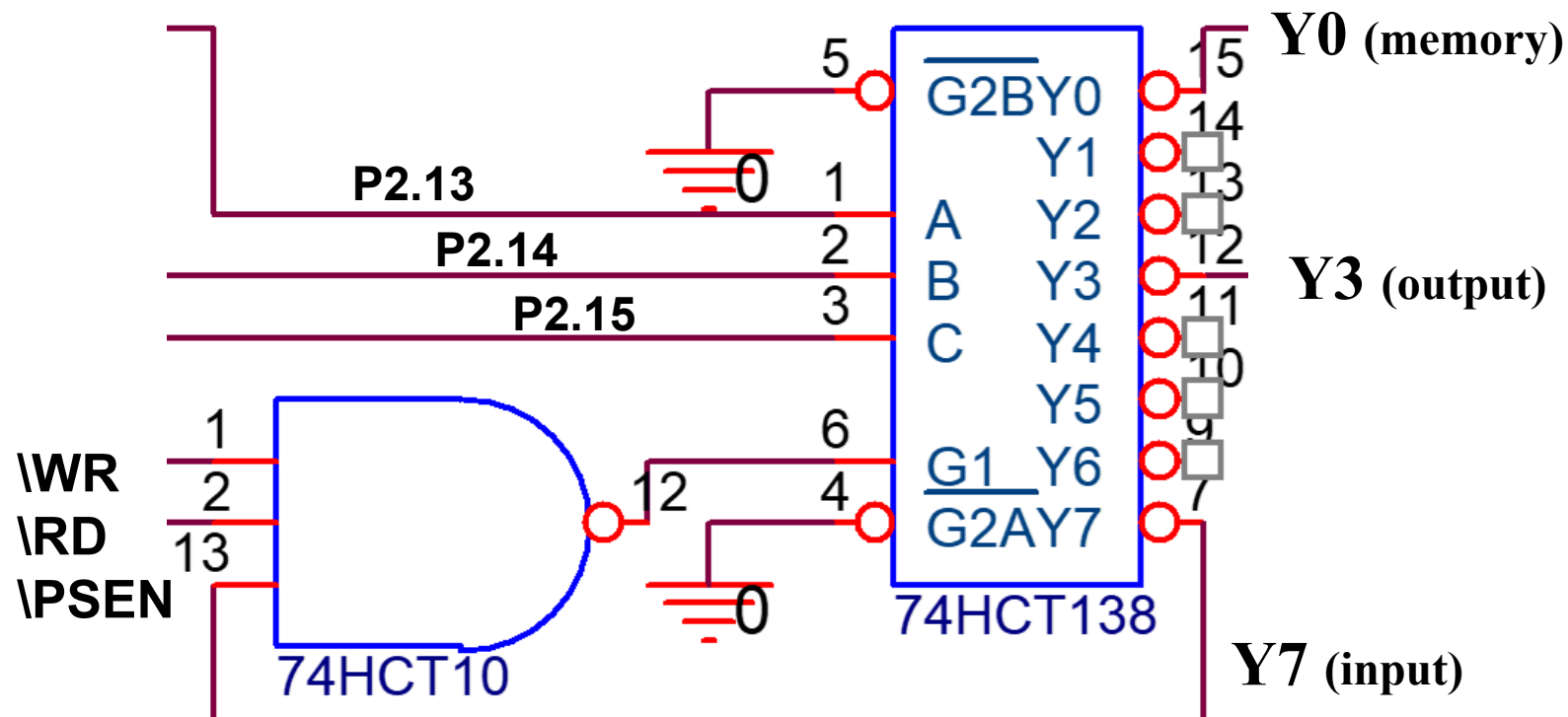


- 8k * 8 external program memory with the low (base) address 0000H
- Output port at address 011XXXXXXXXXXXXX_b
- Input port at address 111XXXXXXXXXXXXX_b

00000.....0_b 0000H
011XX.....X_b Output port
111XX.....X_b Input port

A15 A14 A13	A12 A11... A0
Mem. block selection	8k*8 address lines (13 lines)
+ port selection	+ ports

DCD	A15	A14	A13
Y0	0	0	0
Y1	0	0	1
Y2	0	1	0
Y3	0	1	1
Y4	1	0	0
Y5	1	0	1
Y6	1	1	0
Y7	1	1	1



00000.....0b 0000H
011XX.....Xb Output port
111XX.....Xb Input port

A15 A14 A13 A12 A11... A0
 Mem. block selection | 8k*8 address lines (13 lines)
 + port selection | + ports

DCD	A15	A14	A13
Y0	0	0	0
Y1	0	0	1
Y2	0	1	0
Y3	0	1	1
Y4	1	0	0
Y5	1	0	1
Y6	1	1	0
Y7	1	1	1

Write data into the **74374** register: on the positive front of clock signal CP,
 $Y3\backslash = 0$ and $WR\backslash = 0$

=>**NOR** gate gives us the needed transition from 0 to 1

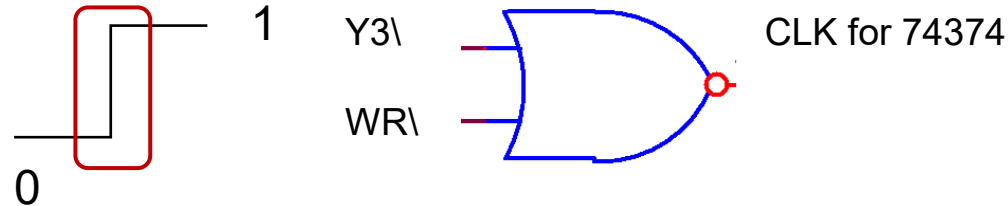
Y3\ WR\ CLK

0 0 1

0 1 0

1 0 0

1 1 0



Read data from keys (**74244** amplifier) (input port): $Y7\backslash = 0$, $RD\backslash = 0$

=>**OR** gate gives us the needed output only when both signals are 0

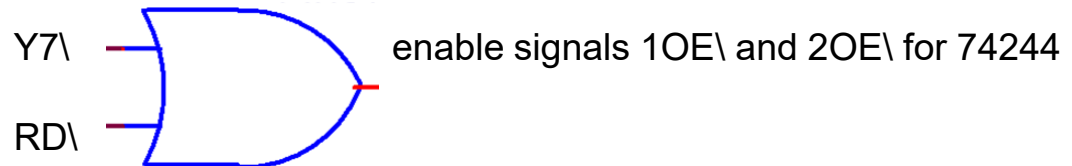
Y7\ RD\ out

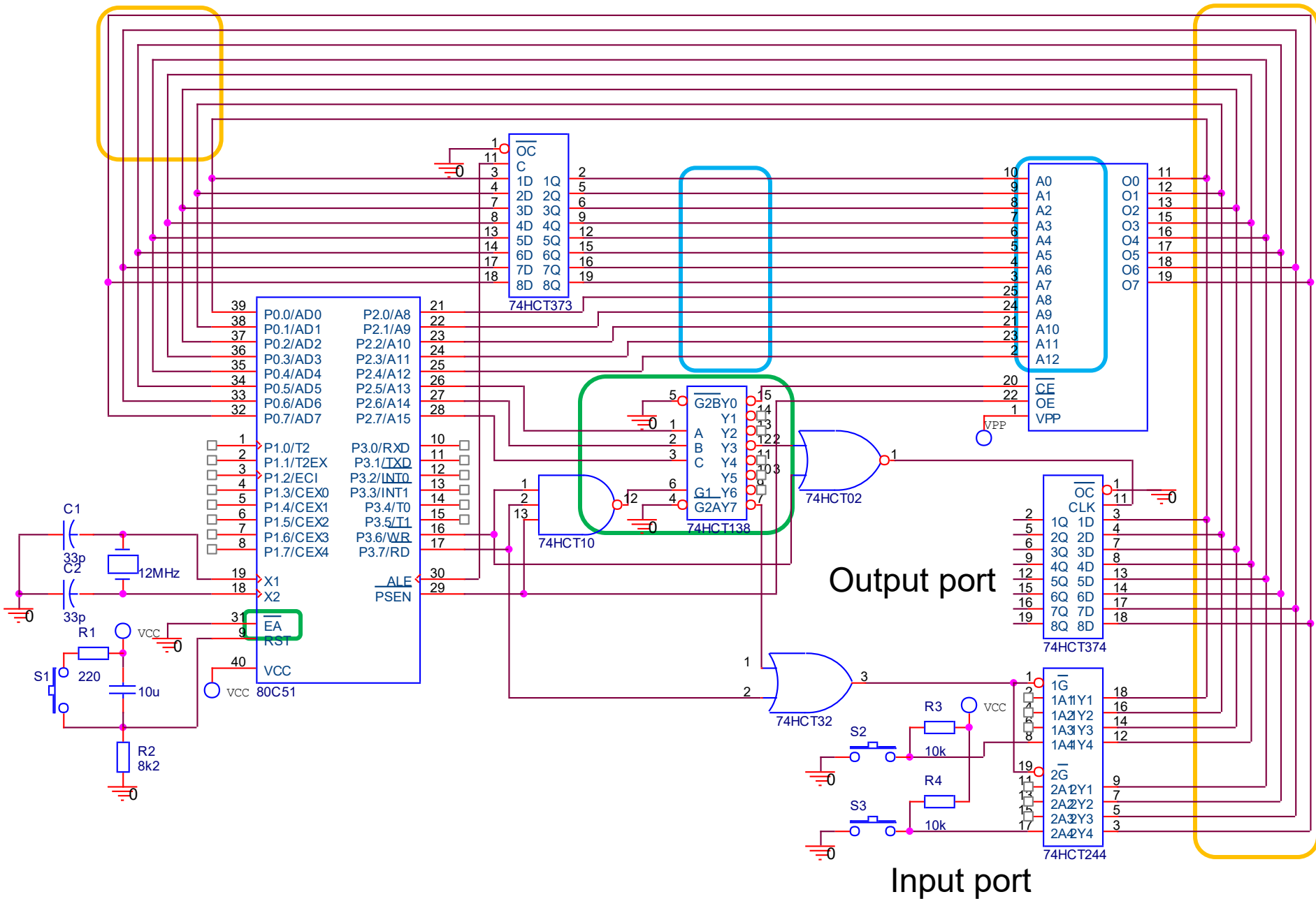
0 0 0

0 1 1

1 0 1

1 1 1





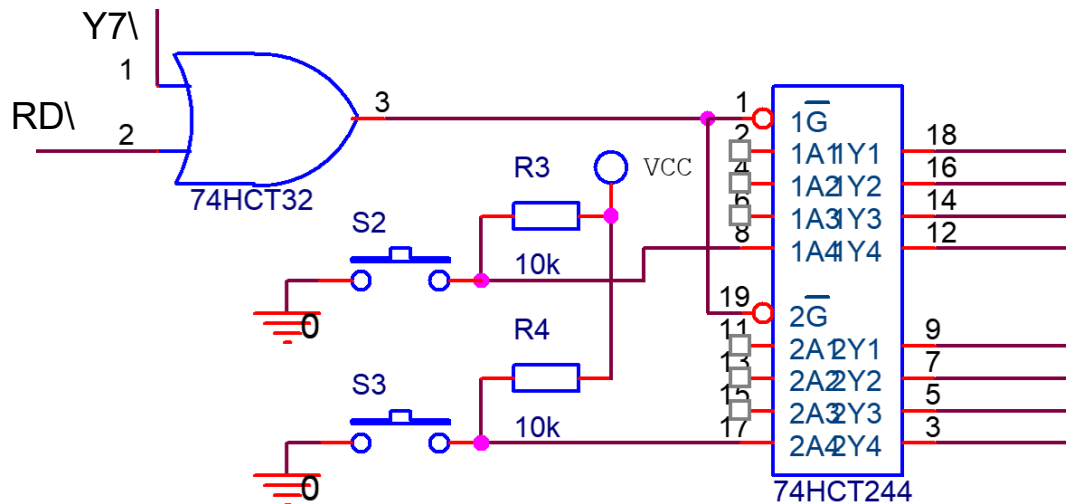
Keys – get data from keys (**74244** amplifier) (input port):

$Y7\backslash = 0$, $RD\backslash = 0$

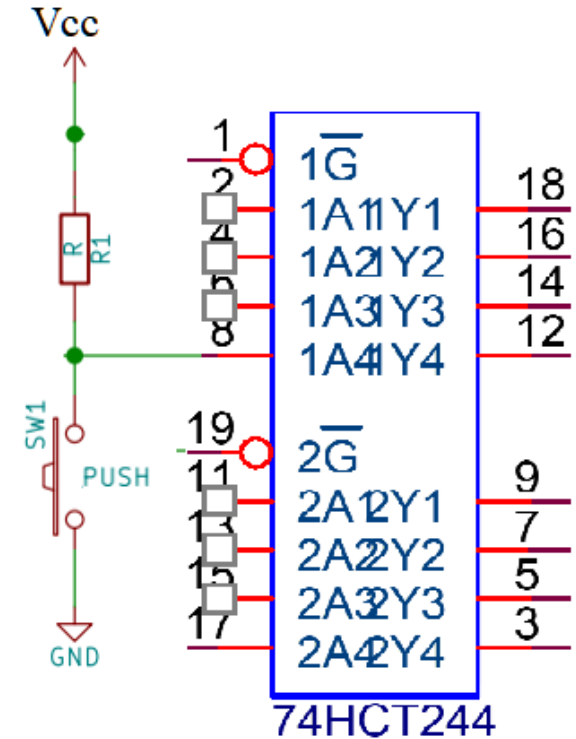
-if SW not pressed -> read “1”

-if SW pressed -> read “0”

read the keys state in a loop – similar to Arduino



Input port



to
Data bus

Design a system based on an 80C51 microcontroller system, with the following blocks:

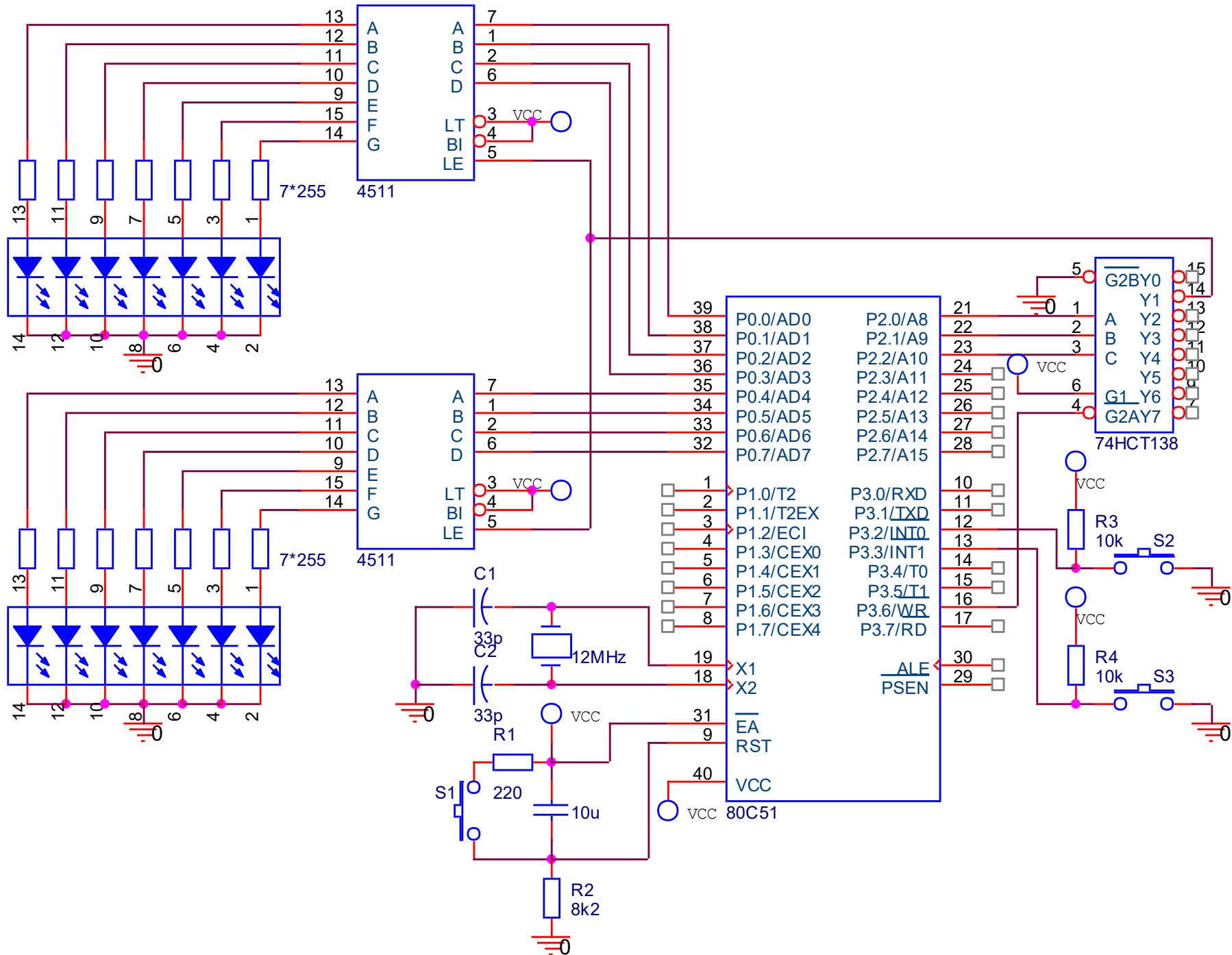
- 12MHz quartz oscillator
- Reset circuit
- Output port at address $XXXXX001XXXXXXX_b$ having connected two 7-segments LED displays using common cathode, with: $V_{LED}=1,7V$ and $I_{LED}=10mA$
- 2 keys connected at interrupt input lines
- *Only internal program memory is used*

Design a system based on an 80C51 microcontroller system, with the following blocks:

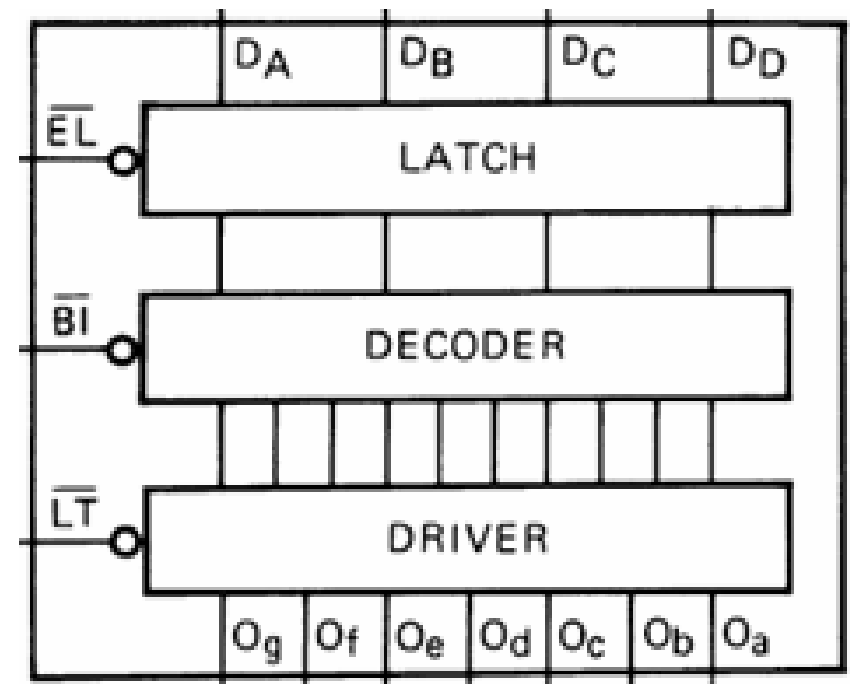
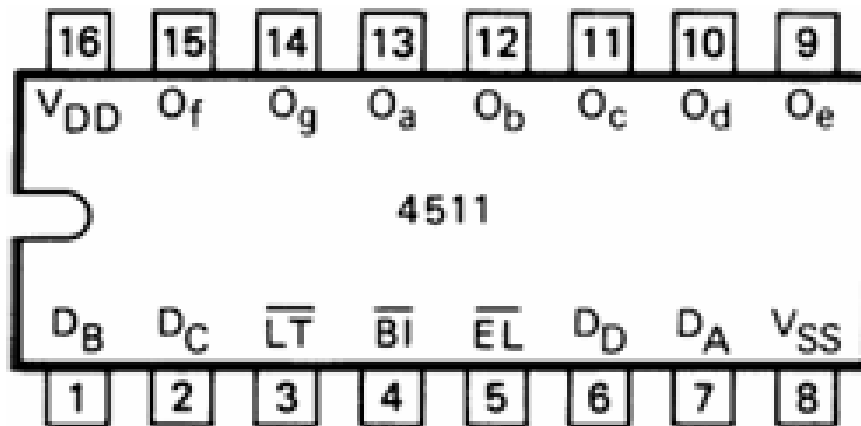
- 12MHz quartz oscillator
- Reset circuit
- Output port at address $\text{XXXXX001XXXXXXXXX}_b$ having connected two 7-segments LED displays using common cathode, with: $V_{\text{LED}}=1,7\text{V}$ and $I_{\text{LED}}=10\text{mA}$
- 2 keys connected at interrupt input lines
- Only internal program memory is used (**EA = 1**)

Why use interrupts?

What addresses should we use (P0 or P2)?

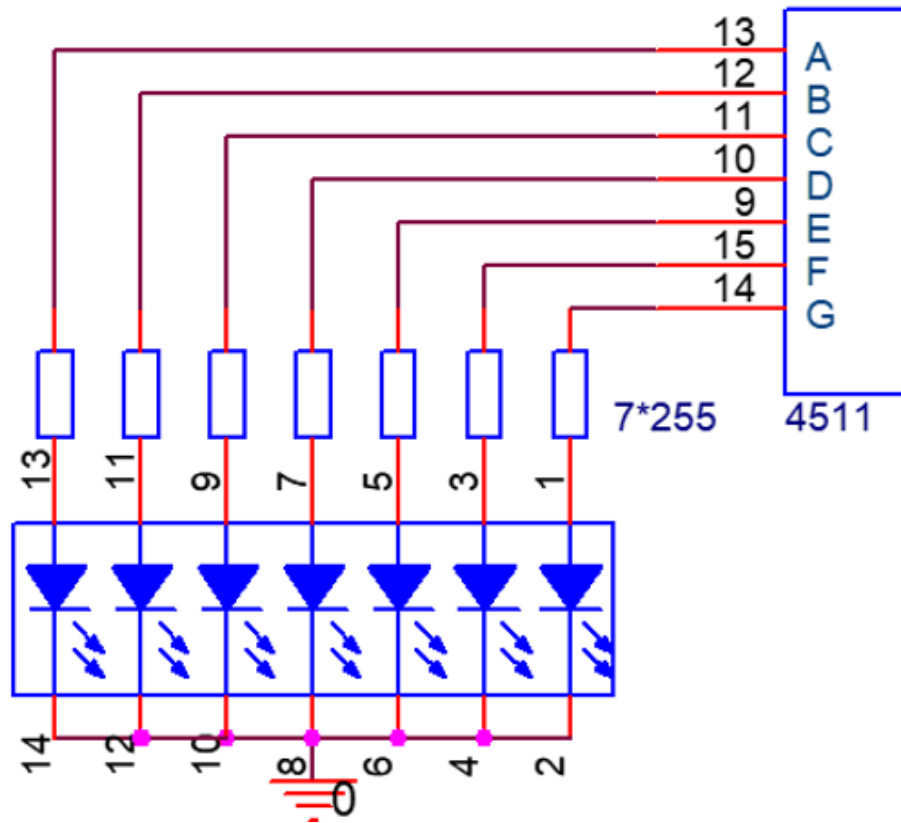


- *remember the functioning from lecture 6:*
 - 4511 - latch, decoder, driver
 - $I_{OH}=10\text{mA} \rightarrow V_{OHmin}=3,60\text{V}, V_{OHtyp}=4,25\text{V}$



Calculus for resistor values (for the LED based display) is given by the formula:

$$R=(V_{OHTYP}-V_{LED})/I_{LED}=(4,25V-1,7V)/10mA=255\Omega$$



Output port at address XXXXX001XXXXXXXXX_b having connected two 7-segments LED displays using common cathode

P2								P0							
A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
X	X	X	X	X	0	0	1	X	X	X	X	X	X	X	X



Y1 on the DCD

